



Reconfigurable Digital FPGA Implementations for Neuromorphic Computing: A Survey on Recent Advances and Future Directions

Farsa, Edris Zaman; Ahmadi, Arash; Keszocze, Oliver

Published in:

IEEE Transactions on Emerging Topics in Computational Intelligence

Link to article, DOI:

[10.1109/TETCI.2025.3551934](https://doi.org/10.1109/TETCI.2025.3551934)

Publication date:

2025

Document Version

Peer reviewed version

[Link back to DTU Orbit](#)

Citation (APA):

Farsa, E. Z., Ahmadi, A., & Keszocze, O. (in press). Reconfigurable Digital FPGA Implementations for Neuromorphic Computing: A Survey on Recent Advances and Future Directions. *IEEE Transactions on Emerging Topics in Computational Intelligence*. <https://doi.org/10.1109/TETCI.2025.3551934>

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal

If you believe that this document breaches copyright please contact us providing details, and we will remove access to the work immediately and investigate your claim.

Reconfigurable Digital FPGA Implementations for Neuromorphic Computing: A Survey on Recent Advances and Future Directions

Edris Zaman Farsa , Arash Ahmadi , *Senior Member, IEEE*, and Oliver Keszocze 

Abstract—Neuromorphic computing represents hardware and software paradigms that emulate neural brain functionalities. Spiking neural networks (SNNs) are a promising brain-inspired computing approach to achieve power efficiency through event-driven processing using discrete asynchronous spikes, making them particularly effective for spatiotemporal data processing. The complex computational nature of SNNs requires intensive calculations and specialized algorithms to ensure accurate performance across different tasks. Hardware accelerators for neuromorphic computing, particularly for SNN implementations, have emerged primarily through field programmable gate arrays (FPGAs) and application-specific integrated circuits (ASICs). FPGAs are especially attractive for neuromorphic computing due to their flexibility, stability, programmability, reconfigurability, and rapid time to market. This research explores top-tier and well-known journal articles from the IEEE Xplore digital library and the Google Scholar databases including IEEE, ACM, Frontiers, Elsevier, Springer, MDPI, Wiley, arXiv, and Nature publishers. In this survey, various energy-efficient and high-performance FPGA implementations of spiking neurons and SNNs are reviewed. The accuracy rates of the implemented SNNs on different applications are investigated. Also, digital hardware optimization techniques for reconfigurable implementations are discussed. The synthesis results from the presented implementations are reported and compared in terms of cost (referring to utilized resources such as Registers/FFs, LUTs, Multipliers, DSP blocks, and Block RAMs), speed, and power/energy consumption. The survey concludes with recommendations for future research directions in FPGA-based neuromorphic computing.

Index Terms—Field programmable gate array (FPGA), hardware implementation, digital design, accelerator, power/energy consumption, neuromorphic computing, spiking neural networks (SNNs), spiking neurons.

I. INTRODUCTION

IN 1965, Gordon Moore, co-founder of Intel, predicted that the number of transistors on a single chip would double

Received 20 November 2024; accepted 20 February 2025. (Corresponding authors: Edris Zaman Farsa; Arash Ahmadi; Oliver Keszocze.)

Edris Zaman Farsa is with the Institute of Applied Microelectronics and Data Technology, Faculty of Computer Science and Electrical Engineering, University of Rostock, 18051 Rostock, Germany, and also with the Department of Computer Engineering, Islamic Azad University (Sanandaj Branch), Sanandaj 6618634683, Iran (e-mail: edriss.zamanfarsa@gmail.com).

Arash Ahmadi is with the Department of Electronics, Carleton University, Ottawa K1S 5B6, Canada (e-mail: arashahmadi3@cunet.carleton.ca).

Oliver Keszocze is with the Department of Applied Mathematics and Computer Science, Technical University of Denmark (DTU), 2800 Lyngby, Denmark (e-mail: olike@dtu.dk).

Recommended for acceptance by A. Lam.

Digital Object Identifier 10.1109/TETCI.2025.3551934

every two years. During the past decades, Moore's law has led impressive progress in the capabilities of data processing. Now, this growth has approximately stopped, indicating a pause in the performance and energy consumption improvements of current processors [1], which demands cutting-edge approaches in data processing to cope with energy and performance issues. As Moore's law scaling nears its end, designing low-energy and high-performance processors is even more challenging than it used to be [2].

In the classical von Neumann architecture, the memory and processor are separated from each other. Therefore, the data transmission between them for storage and computation is costly in terms of energy, power, and latency, and is defined as the von Neumann bottleneck or memory wall. To this end, the concentration of the recent research has focused on designing brain-inspired computer architectures like neuromorphic computing systems to overcome the memory/CPU bottleneck in conventional computers. The bottleneck caused by the processor/memory separation is lessened by collocating the processing and memory as a single neuron unit in neuromorphic computers [3], [4].

There are considerable research initiatives underway throughout the world to build a fundamentally new computer paradigm for artificial intelligence (AI) applications based on biological and brain-inspired principles [5], [6], and [7]. In an era when CMOS digital computing approaches are reaching physical constraints, neuromorphic computers, which are considered as non-von Neumann computing systems, are becoming increasingly important [8].

Neuromorphic systems mimic neural structures for energy-efficient and high-performance computations employed in industry and academia [8]. Neuromorphic computing refers to both software and hardware, which implements brain-inspired models, often in the form of spiking neurons or spiking neural networks (SNNs) [9], [10], and [11]. Event-driven specification of the SNNs has resulted in extremely efficient computing systems with collocated memory and processing units, higher parallelism, and lower energy consumption [12].

The well-known neuromorphic chips are Loihi from Intel corporation [13], TrueNorth from IBM corporation [14], SpiN-Naker created by the University of Manchester [15], SpiN-Naker2, a joint project from Technische Universität Dresden and the University of Manchester [16], ODIN, designed by the Catholic University Louvain [17], NeuroGrid, presented from

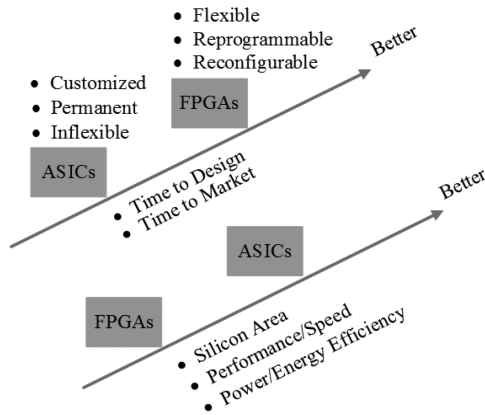


Fig. 1. A comparative overview on FPGAs and ASICs.

Stanford University [18], BrainScaleS-2, made at Heidelberg University [19], and DYNAPs, built at the Zurich Institute of Neuroinformatics [20]. Digital hardware implementations of spiking neurons and networks for various tasks have been presented on application-specific integrated circuits (ASICs) [13], [14], [17], [21], [22], [23], [24], [25] and field programmable gate arrays (FPGAs) [9].

Compared to FPGAs, ASICs are more efficient in silicon area, energy/power, and performance. Because ASICs offer better power/energy efficiency and performance than FPGAs, they play a key role in high-speed power-sensitive devices such as laptops, tablets, smartphones, iPads, and other battery-powered systems. An ASIC is a type of permanent custom chip designed for a specific application in which the design cannot be modified and updated after it has been manufactured. Hence, ASICs are inflexible. ASICs also have a long time to market and development time, which increase manufacturing overheads and make them expensive. FPGAs, on the other hand, are notable hardware platforms for digital implementations of AI algorithms and brain-inspired computing like neuromorphic systems and deep learning algorithms due to being flexible, reprogrammable, and reconfigurable, having short design and development times, and offering a short time to market [5], [6], [26], [27], [28], and [29]. The flexibility of FPGAs is a very important advantage, allowing iterative modifications and post-implementation upgrades. This means the FPGAs can be reprogrammed and reconfigured even after deployment. In domains like AI, where algorithms and structures are always evolving, this adaptability is highly crucial. Furthermore, FPGA design and development cycles are remarkably shorter than those of ASICs. While ASICs need to be extensively validated and tested before considering sending them out for manufacturing, which is a very time-consuming process, FPGAs benefit from being directly programmed and synthesized using hardware description languages (HDLs) or even high-level synthesis (HLS) tools. This speedy time to market is especially helpful in fast-moving industries where having the ability to quickly adjust to new developments in technology or consumer needs can give an advantage over competitors. Fig. 1 displays a general overview of pros and cons for FPGAs and ASICs.

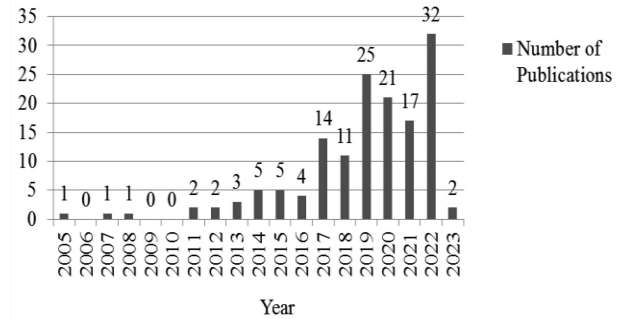


Fig. 2. The distribution of the numbers of the surveyed journal articles in our work based on each year from 2005 to the early of 2023.

This survey reviews reconfigurable computing on various FPGA devices for neuromorphic computing systems, focusing on spiking neurons and SNNs. The implemented SNNs work with different learning algorithms in diverse applications. Finally, various upcoming prospects for research in this field will be discussed. The rest of this article is organized as follows. Section II explains the methodology of the research for collecting and analyzing the papers considered in this survey. Section III explores various existing FPGA implementations of spiking neurons and SNNs. Section IV discusses algorithm/hardware optimization techniques and proposes a roadmap for future research and directions. Section V concludes the survey.

II. METHODOLOGY AND STRATEGY

The research method in this survey involved searching journal articles on the IEEE Xplore digital library and the Google Scholar databases by using hybrid keywords comprising “FPGA implementation of spiking neurons”, “FPGA implementation of spiking neural networks”, “FPGA implementation of neuromorphic computing”, “digital implementation of spiking neurons”, “digital implementation of spiking neural networks”, “spiking neurons on FPGAs”, “spiking neural networks on FPGAs”, “neuromorphic computing on FPGAs”, “FPGAs for neuromorphic computing”, “hardware implementation of spiking neurons”, “hardware implementation of spiking neural networks”, “FPGA accelerator for spiking neurons”, and “FPGA accelerator for spiking neural networks”. Specifically, the publications in conferences and book chapters were excluded due to the focus on comprehensive and salient research presented in journal articles. At the start, around 180 papers were extracted. After a thorough investigation, the number of papers for reviewing was limited to 146. The publishers that have been included for exploration are IEEE, ACM, Frontiers, Elsevier, Springer, MDPI, Wiley, arXiv, and Nature. The journal papers reviewed in this work are from 2005 until early 2023. Figs. 2 and 3 show the distribution of the papers reviewed in this work based on each year and the publishers, respectively. Fig. 2 shows that research on using reconfigurable FPGA implementations of neuromorphic systems has risen. As can be seen from 2019 to 2021, the number of research works has slightly fallen, likely

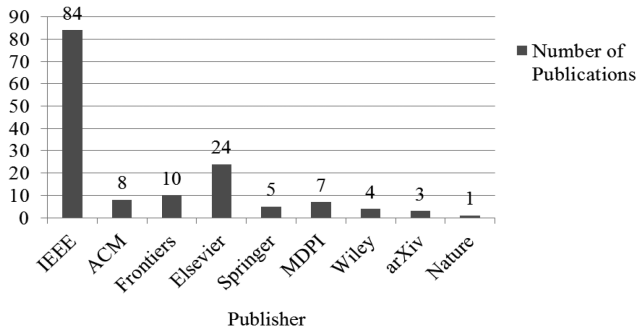


Fig. 3. The distribution of the numbers of the surveyed journal articles in our work based on each publisher from 2005 to the early of 2023.

due to the COVID-19 pandemic. Since 2022, it has taken an upward trend again. This shows the importance of the research area and predicts its development in the coming years. As seen in Fig. 3, the IEEE publisher is considerably more prominent than others, as it covers the most research in the field.

III. FPGA IMPLEMENTATIONS OF SPIKING NEURONS AND SPIKING NEURAL NETWORKS

This section presents a background on FPGA design and implementations of spiking neurons, learning algorithms, and SNNs. Also, efficient FPGA digital implementation techniques in Verilog HDL and VHDL will be discussed. These techniques lead to a decrease in computational complexity in order to obtain low-cost, low-power, and high-speed implementations on FPGAs.

There are approximation techniques for efficient digital implementations. These methods are categorized as using shift/add operations instead of multipliers, coordinate rotation digital computer (CORDIC), reducing the number of bits, and piecewise linear model. Although the computational costs and utilized resources have been reduced by employing these techniques, it is important to realize that there is a trade-off between accuracy, power/energy consumption, performance, and utilized hardware resources. Here, we give a short overview of the aforementioned methods:

- If there is a coefficient constant multiplied by a variable, the constant parameter can be approximated based on powers of 2. This approach helps to eliminate the multipliers as they are costly blocks and uses shift/add operations instead of them.
- CORDIC is digital hardware with resource-efficient iterative shift/add blocks that applies rotations to approximate functions like exponential, logarithmic, hyperbolic, and trigonometric.
- Reducing the number of bits refers to a technique in which the number of bits in registers is reduced, while causing a small reduction in accuracy and keeping an acceptable error.
- A piecewise linear model is a mathematical approximation that divides curved segments into straight lines to simplify the digital computations on FPGAs. The piecewise linear

technique is utilized when there are two or more variables multiplied and make a curve shape.

The common synthesizers used for synthesizing digital hardware are Xilinx ISE, Xilinx Vivado, and Altera Quartus. According to the synthesized results there are some significant parameters that describe the digital hardware. These parameters are the number of slice registers, the number of slice LUTs, the number of block RAMs, the number of DSP blocks (e.g., DSP48E1 on Xilinx devices), the maximum frequency, and the clock period.

In FPGA synthesis, the number of DSPs indicates the number of expensive multipliers that incur costs and occupy silicon area. By using shifters and adders to eliminate the multipliers, the number of required DSPs considerably decreases and, hence, helps to reduce cost and resource utilization. Also, the number of slice registers and the number of slice LUTs would be increased as employing shifters and additions require more utilized digital resources and clock cycles. Moreover, the maximum frequency (MHz) increases and clock period (nanosecond) decreases when multipliers are eliminated. Both maximum frequency and clock period show the speed of the digital hardware designed on FPGAs.

In this survey, Verilog HDL and VHDL are the hardware description languages used for FPGA implementations in the reviewed papers. Moreover, Xilinx ISE, Xilinx Vivado, and Altera Quartus are the tools employed for synthesizing digital hardware and extracting the resource utilization reports. Each type of FPGA board contains different available resources like slice registers, slice LUTs, RAMs, and DSP blocks [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41], [42], [43], [44], [45], [46], [47], [48], [49], [50], [51], [52], [53], [54], [55], [56], [57], [58], [59], [60], [61], [62], [63], [64], [65], [66], [67], [68], [69], [70], [71], [72], [73], [74], [75], [76], [77], [78], [79], [80], [81], [82], [83], [84], [85], [86], [87], [88], [89], [90], [91], [92], [93], [94], [95], [96], [97], [98], [99], [100], [101], [102], [103], [104], [105], [106], [107], [108], [109], [110], [111], [112], [113], [114], [115], [116], [117], [118], [119], [120], [121], [122], [123], [124], [125], [126], [127], [128], [129], [130], [131], [132], [133], [134], [135], [136], [137], [138], [139], [140], [141], [142], [143], [144], [145].

A. Implementing Spiking Neurons on FPGAs

In this section, low-cost and high-speed FPGA implementations of various spiking neurons and their families will be reviewed. The concrete neuron models discussed in this section are Izhikevich (Izhi) model [30], [31], [32], [33], [34], [35], adaptive exponential integrate and fire (AdEx) model [36], [37], [38], [39], [40], leaky integrate and fire (LIF) model [41] and [42], rotate and fire model [43] and [44], FitzHugh-Nagumo (FHN) model [45] and [46], Hindmarsh-Rose (HR) model [47], [48], [49], [50], Morris-Lecar (ML) model [51], [52], [53], Hodgkin-Huxley (HH) model [54], [55], [56], Wilson model [57], [58], [59], cellular models [60], uniform-segmented models [61], quadratic-based model [62]. Fig. 4 displays a general structure for a spiking neuron, showing the processing flow of neural activity.

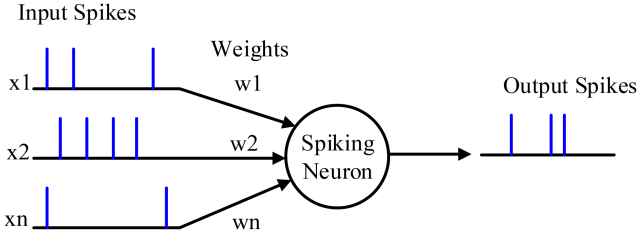


Fig. 4. Graphical representation of a spiking neuron, including input and output spikes.

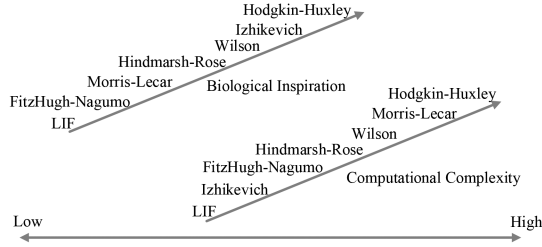


Fig. 5. Illustration of the complexity cost and biological plausibility for spiking neurons.

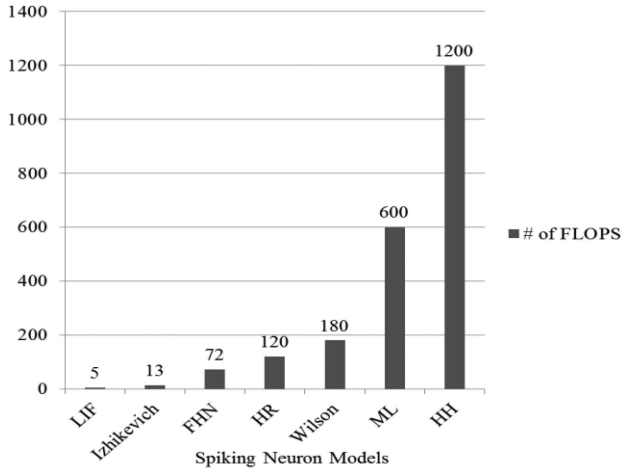


Fig. 6. Visualization of the number of FLOPS for each spiking neuron.

The computational complexity and biological inspiration features for different spiking neuron models have been illustrated in Fig. 5. Complexity is a multifaceted concept and can be interpreted in various ways, depending on the context in which it is being discussed. In our work, however, we define complexity as computational (algorithmic) complexity, which measures the efficiency of algorithms and software and it is typically expressed by order of the algorithms. In the context of hardware, on the other hand, we can measure complexity in terms of implementation costs such as the number of components, circuit delay, and power consumption. Fig. 6 quantifies and compares the computational complexity of spiking neurons through a neuro-computational specification called approximate number of floating-point operations (# of FLOPS). The FLOPS parameter involves addition, multiplication, and other operations required to simulate the model for one millisecond.

TABLE I
SYNTHESIS REPORTS FOR SEVERAL FPGA IMPLEMENTATIONS OF THE IZHKEVICH NEURON

Ref.	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
[30]	493	617	241.937	X. Virtex-II XC2VP30
[31] ¹	551	1268	28	X. Virtex-II XC2VP30
[31] ²	408	459	240	X. Virtex-II XC2VP30
[32]	-	-	-	X. Virtex-5 XC5VLX30T
[33]	274	309	224	X. Zynq-7000 SoC ZC702
[34]	41	364	-	A. DE0 Cyclone III
[35]	414	649	103	X. Spartan-6 Lx9

¹original, ²approximate, FFs: flip-flops

1) *Digital Izhikevich Neuron*: An FPGA implementation of the Izhikevich neuron model is presented in [30]. For efficient digital implementation, first, a piecewise linear method has been employed to simplify the model, and then shift/add operations instead of the multipliers have been used. The piecewise linear technique is employed using two, three, and four linear segments for approximating nonlinear terms.

In [31], the original noisy Izhikevich neuron and multiplierless noisy Izhikevich neuron implementations on an FPGA are discussed. To understand the multiplierless design, a set of hyperbolic functions has been considered for removing the nonlinear terms in the noisy Izhikevich neuron. A hyperbolic function is rewritten based on the addition and subtraction of two exponential terms. Each exponential function is replaced by a 2^x function. Each 2^x term is implemented using shift modules. Therefore, a hyperbolic function is converted to a set of exponential functions, which are simply realized by shift and add modules.

In [32], the fractional-order Izhikevich neuron and its FPGA realization are elaborated. The implementation was performed by employing non-uniform segmentation in the form of second-degree polynomial approximations with an address remapping technique. The combinational logic was used for multiplierless implementation. The number of slices reported from the synthesis process was 1245. Moreover, there were no occupied silicon areas for multipliers, RAMs, and DSPs.

In [33], the Izhikevich neuron has been digitally implemented on an FPGA. The quadratic polynomial in Izhikevich neuron has been replaced by the *cosh* function to reach cost-efficient and high-throughput digital hardware. The *cosh* function is calculated by adding/subtracting two exponential terms. The exponential functions could be approximated based on powers of 2 terms, which can be implemented by shifts. Thus, the *cosh* function has been expressed by shift and add blocks.

In [34], a modified Izhikevich neuron model has been realized on an FPGA board. The proposed Izhikevich neuron approximates the constant coefficients on the base of powers of 2, which are implemented by shifts instead of multipliers. In [35], an FPGA implementation of the Izhikevich neuron using stochastic computing is proposed. In this case, a stochastic multiplier is designed using a stochastic number generator as the foundation of stochastic computing. The Izhikevich neuron was described and digitally implemented by the designed multiplier. Throughout this paper, we abbreviate Xilinx and Altera as X. and A., respectively. Table I summarizes the synthesis

TABLE II
SYNTHESIS REPORTS FOR VARIOUS FPGA IMPLEMENTATIONS OF THE AdEx NEURON

Ref.	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
[36]	388	379	190	X. Virtex-II XC2VP30
[37]	829	1221	134.3	X. Spartan-6 XC6SLX9
[38]	244	314	350.4	X. Virtex-6 ML605
[39] ¹	956	1753	34	X. Virtex-II XC2VP30
[39] ²	270	643	196	X. Virtex-II XC2VP30
[40]	292	363	500	X. ZCU102

¹original, ²approximate

results from several FPGA implementations of the Izhikevich neuron.

2) *Digital AdEx Neuron*: In [36], the AdEx neuron on an FPGA is discussed. For cheaper FPGA implementations, piecewise linear and piecewise linear exponential methods were used to approximate the AdEx neuron. Piecewise linear exponential method is a more efficient technique for FPGA implementation. In the piecewise linear exponential method, the exponential function was converted to a term based on the power of 2 that can be implemented by shift operations. Also, the constant coefficients are modified in a way so that shift/add operations are employed instead of multipliers. In [37], a CORDIC-based FPGA implementation of the AdEx neuron is introduced. Also, the multiplication of a variable by a constant was replaced with add and shift operators. In [38], digital implementation of a uniform piecewise linear segmentation approach to approximate the nonlinear part of the AdEx neuron, named uniform-segmented AdEx model, is explained. The multiplications are replaced with shifts and additions. In [39], digital implementations of the original AdEx and power-2 based AdEx neurons are suggested. In the power-2 based AdEx model, the exponential function has been approximated by a power of 2 based function. Power-2 based AdEx is a multiplierless version of the original AdEx neuron, and it only uses shift and add modules.

In [40], stochastic computing (SC) for an efficient hardware implementation of the AdEx neuron, called SC-AdEx, is employed. This is the first attempt to apply stochastic computing to a spiking neuron. The arithmetic computations were done based on stochastic bit-streams, which are directed to the low-power high-speed digital neuron. In contrast to the traditional binary computing paradigm, stochastic computing performs arithmetic computation and encodes real numbers by random binary bit-streams. Unlike using the expensive multiplications and exponential terms in the AdEx neuron, only AND operations and ADD operations were used in SC-AdEx, which help to achieve a cost-efficient and high-performance hardware implementation. The synthesized hardware reports 863 mW power consumption.

Table II presents the synthesis results from various FPGA implementations of the AdEx neuron.

3) *Digital LIF Neuron*: In [41], a digital implementation of the LIF neuron is studied. The model has been modified by removing a costly multiplier with add and shift operations. In [42], an LIF neuron model has been digitally implemented employing the template-scaling-based exponential function approximation (TS-EFA) technique. In this case, exponential functions have

TABLE III
SYNTHESIS REPORTS FOR DIFFERENT FPGA IMPLEMENTATIONS OF THE LIF NEURON

Ref.	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
[41]	46	56	412.371	X. Virtex-6 ML605
[42] ¹	-	74	500	X. Virtex-7
[42] ²	-	76	500	X. Virtex-7

¹EXP_FIX, ²EXP_VAR

been approximated by the template-scaling technique. This technique makes use of a template consisting of a collection of previously calculated exponential function values over a given time period. To determine the function values throughout the whole input range, the template is scaled repeatedly. TS-EFA has been implemented in two forms, known EXP_FIX and EXP_VAR. EXP_FIX refers to a fixed time constant-based TS-EFA module and EXP_VAR refers to a TS-EFA module that treats the time constant as a variable. The multiplication that appeared in the neuron's equation is performed by a left-shift module employing just one built-in DSP slice.

Table III presents the synthesis outcomes from different FPGA implementations of the LIF neuron.

4) *Digital Rotate-and-Fire Neuron*: In [43], the rotate-and-fire digital spiking neuron in which shift registers are connected in a wired system is studied. The digital neuron can produce spike trains with different spike patterns and display relevant bifurcations by changing the wiring pattern between the registers. The synthesis report from Xilinx XC3S400 FPGA included 5 slice registers, one 16×10 -bit ROM, and one 16×4 -bit ROM. There were no reports for LUTs, DSP blocks, and maximum speed. In [44], it is suggested to create a generalized rotate-and-fire digital spiking neuron (GRN) model using a straightforward asynchronous sequential logic circuit. A Xilinx FPGA Virtex-5 XC5VLX100 has been employed to implement the GRN model and its learning. The number of required slices used for the digital implementation was 414.

5) *Digital FitzHugh-Nagumo Neuron*: In [45], a multiplierless FPGA implementation of the FitzHugh-Nagumo neuron by employing shifters and adders based on the piecewise linear technique is explored. In [46], a multiplierless FitzHugh-Nagumo neuron to display bursting behavior in neural activities, named the multiplierless burst-mode FitzHugh-Nagumo model, is described. Time-varying functions were added to the original neuron equations and then approximated by linear functions. The benefit of the time-varying factor is to control the frequency of the neuron model to generate bursting behavior. For the FPGA implementation of the model, the hyperbolic functions were expressed based on addition/subtraction of exponential functions and then the exponential terms were converted to the powers of 2 that could be expressed by shift operators for obtaining low-cost high-speed digital implementation. Also, each constant coefficient was approximated by shift and add modules.

The synthesis results from distinct FPGA implementations of the FitzHugh-Nagumo neuron have been presented in Table IV.

6) *Digital Hindmarsh-Rose Neuron*: In [47], two-coupled Hindmarsh-Rose neurons have been digitally implemented. The implementation method is the same as used in [45]. In [48],

TABLE IV
SYNTHESIS REPORTS FOR DISTINCT FPGA IMPLEMENTATIONS OF THE
FITZHUGH-NAGUMO NEURON

Ref.	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
[45]	526	1085	-	X. Virtex-II XC2VP30
[46] ¹	641	892	64	X. Virtex-II
[46] ²	457	912	175	X. Virtex-II

¹original, ²approximate

TABLE V
SYNTHESIS REPORTS FOR SOME FPGA IMPLEMENTATIONS OF THE
HINDMARSH-ROSE NEURON

Ref.	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
[47]	451	659	81.2	X. Virtex-II
[48]	1589	-	-	A. DE2-115
[49]	611	1524	73.18	X. Spartan-6 XC6SLX9
[50]	469	-	139	X. xc7z020clg484-2

TABLE VI
SYNTHESIS REPORTS FOR DIVERSE FPGA IMPLEMENTATIONS OF THE
MORRIS-LECAR NEURON

Ref.	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
[51]	618	3616	135	X. Virtex-II XC2VP30
[52]	1040	2572	155	X. Spartan-6 xc6slx100
[53]	830	1357	378.07	X. Kintex-7

FPGA implementations of a piecewise linear approximation of the fractional order Hindmarsh-Rose (HR) neuron model and its two-coupled model are propounded. All multipliers are replaced by the shift and add modules. In [49], a digital implementation of the 2D Hindmarsh-Rose neuron on an FPGA is investigated by using piecewise linear approximation and applying shifts and additions instead of multiplication. The piecewise linear technique was used by approximating the nonlinear terms with two, three, and four linear segments. In [50], a digital multiplierless Hindmarsh-Rose (HR) neuron based on the nullcline-characteristics-based piecewise linear (NC-PWL) method is explained. NC-PWL approximated the nonlinearities of the HR neuron model with the fewest number of line segments feasible. Shift modules were used for digital multiplierless implementation.

Table V illustrates the digital synthesis of the FitzHugh-Nagumo neuron from FPGA implementations.

7) *Digital Morris-Lecar Neuron*: In [51], the multiplierless piecewise linear approximation methods on the basis of the two-coupled Morris-Lecar neurons for FPGA implementation are provided. The hyperbolic functions presented in the neuron equations have been calculated by the exponential terms and the exponential functions have been designed based on the powers of 2. In [52], the division-based Morris-Lecar neuron on an FPGA device was discussed. By this implementation, the hyperbolic terms have been approximated by a divider and the exponential functions realized by 2^x that would be implemented using shifters and adders. In [53], hardware acceleration of a very accurate digital version of the Morris-Lecar neuron was suggested. The method uses the CORDIC algorithm to produce a fixed-point implementation that is not only extremely accurate but also uses little digital hardware.

TABLE VII
SYNTHESIS REPORTS FOR VARIANT FPGA IMPLEMENTATIONS OF THE
HODGKIN-HUXLEY NEURON

Ref.	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
[54]	2840	5660	85	X. Virtex-II XC2VP30
[55]	480	2344	200	X. Virtex-4 XC4VLX15
[56]	94	19246	-	-

Table VI displays the synthesis outcomes for the Morris-Lecar neuron from diverse FPGA implementations.

8) *Digital Hodgkin-Huxley Neuron*: A digital multiplierless Hodgkin-Huxley neuron is proposed in [54]. For simplifying the Hodgkin-Huxley model, piecewise linear functions were employed for nonlinear functions of the neuron, and all multiplications were implemented based on the power of 2 terms by shifters and adders. To implement the hyperbolic terms in the neuron model, exponential functions have been used and then converted into powers of 2, which can be expressed through shifts.

In [55], the Hodgkin-Huxley neuron was implemented by using base-2 functions on an FPGA. A set of multiplierless terms on the base of the powers of 2 was employed for high-speed and low resource utilization digital implementation of Hodgkin-Huxley model. The nonlinear functions were converted to a set of base-2 terms. Then, all the based-2 functions were digitally implemented by shift and add blocks. In [56], the nonlinear equations of Hodgkin-Huxley (HH) neuron model are linearized and then the linear Hodgkin-Huxley neuron has been implemented on an FPGA.

In Table VII, the synthesis results of variant FPGA implementations for the Hodgkin-Huxley neuron are presented.

9) *Digital Wilson Neuron*: In [57], the digital coupled Wilson neuron model has been suggested. For efficient hardware design, the Wilson equation has been approximated by five linear lines that refer to the piecewise linear method. Furthermore, simple arithmetic operations, such as shift and add, have been used to replace nonlinearities like multiplications and exponentiations. In [58], the 2D Wilson neuron is approximated by the piecewise linear technique to go through an efficient digital design. The piecewise linear approximation method employs 5 linear segments to approximate the nonlinear Wilson model and then, the constant coefficient multipliers have been substituted by using add and shift modules. In [59], the Gaussian Wilson-Cowan neuron as a hybrid model has been implemented. This model includes square and exponential functions. These functions have been approximated based on the methods introduced in [30] and [36]. The square function has been approximated using the piecewise linear technique, which approximates the curve based on first-order linear lines introduced in [30], and the exponential function has been approximated by a power of 2 term introduced in [36].

The digital synthesis results of the Wilson neuron from different FPGA implementations have been reported in Table VIII.

10) *Digital Cellular Izhikevich, FitzHugh-Nagumo, and Hindmarsh-Rose Neurons*: In [60], a reconfigurable cellular spiking neuron model is suggested. The cellular model was applied to the Izhikevich and FitzHugh-Nagumo neurons as

TABLE VIII
SYNTHESIS REPORTS FOR DIFFERENT FPGA IMPLEMENTATIONS OF THE WILSON NEURON

Ref.	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
[57]	520	737	145	-
[58]	-	617	191.81	X. Virtex-II XC2VP30
[59]	535	1006	238.3	A. Stratix IV EP4SGX530KH

TABLE IX
SYNTHESIS REPORTS FOR VARIOUS FPGA IMPLEMENTATIONS OF THE CELLULAR SPIKING NEURONS TAKEN FROM [60]

Neuron Model	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
Izhikevich	265	274	254.26	X. Virtex-II XC2VP30
FHN	370	373	248.14	X. Virtex-II XC2VP30
HR	425	435	309.37	X. Virtex-II XC2VP30
Izhikevich	187	223	597.58	X. Kintex-7 XC7K325T
FHN	256	414	563.88	X. Kintex-7 XC7K325T
HR	364	416	663.10	X. Kintex-7 XC7K325T

TABLE X
SYNTHESIS REPORTS FOR SEVERAL FPGA IMPLEMENTATIONS OF THE UNIFORM-SEGMENTED SPIKING NEURONS TAKEN FROM [61]

Neuron Model	Reg./FFs	LUTs	Speed (MHz)	FPGA Device
Izhikevich	199	230	285	X. Virtex-6 XC6VLX240T
FHN	193	216	335	
HR	260	265	340	
Izhikevich	200	225	289	X. Virtex-5 XC5VFX130T
FHN	194	211	312	
HR	261	267	300	
Izhikevich	184	234	194	X. Spartan-6 XC6SLX100
FHN	195	214	204	
HR	260	269	201	
Izhikevich	218	335	222	X. Virtex-II XC2VP30
FHN	199	314	211	
HR	268	304	228	

two-dimensional neural models and to the Hindmarsh-Rose neuron as a three-dimensional neural model. Employing shifts instead of constant multipliers is considered. The cellular Izhikevich, cellular FitzHugh-Nagumo, and cellular Hindmarsh-Rose neurons were synthesized and implemented on two different devices.

Table IX categorizes the hardware synthesis outcomes from various FPGA implementations of the cellular spiking neurons presented in [60].

11) *Digital Uniform-Segmented Izhikevich, FitzHugh-Nagumo, and Hindmarsh-Rose Neurons*: In [61], a uniform piecewise linear segmentation technique to approximate the nonlinear terms in the Izhikevich, FitzHugh-Nagumo, Hindmarsh-Rose neuron models, and a calcium-based synaptic plasticity model is propounded. The calcium-based synaptic plasticity is able to generate the spike timing dependent plasticity (STDP) learning curves. Each constant multiplication in the mathematical models can be eliminated by shift and add blocks. According to the various numbers of segments (32, 64, and 128 segments) in the approximation method, the neural models were implemented on different devices.

Table X shows FPGA synthesis of the uniform-segmented spiking neurons with 128 segments for each neuron model on various Xilinx devices presented in [61].

12) *Digital Quadratic-Based Spiking Neuron*: In [62], a quadratic-based digital spiking silicon neuron (DSSN) has been presented and then modified based on hyperbolic functions, which can be approximated by a set of powers of two (2^x). This helps to compute the digital neuron by shifts and additions. The Xilinx Spartan-3 FPGA synthesis of the original DSSN illustrates 1150 slice registers, 1050 slice LUTs, and 63 MHz speed and the synthesis of the modified DSSN displays 445 slice registers, 953 slice LUTs, and 168 MHz speed.

B. Spiking Neural Networks and Learning Algorithms on FPGAs

This section reviews various SNNs with their learning algorithms implemented on FPGAs for different applications. Due to the capacity of FPGAs to achieve real-time processing, energy efficiency, and low latency capabilities, FPGAs have attracted a lot of attention when it comes to the implementation of SNNs. Because of this, scientists have looked into a variety of architectural layouts, optimization methods, and deployment approaches to take advantage of the built-in parallelism and reconfigurability of FPGAs to accelerate SNN computations. The reviewed papers in this section demonstrate the variety of strategies used by researchers to tackle the difficulties in implementing SNNs on FPGA platforms. These strategies include hardware resource optimization, neural network structures, and effective learning algorithms specifically designed for FPGA-based implementations.

We hope to provide a thorough grasp of FPGA-based SNN implementations by analyzing the results and insights from these investigations. This will help to illuminate the opportunities and challenges in this quickly developing field of research.

Fig. 7 illustrates the programming languages and tools used for FPGA implementations, particularly for SNNs, covering the sequence from software simulation to digital FPGA implementation. According to the fact that which application is considered for being processed by an SNN, the input data are selected from standard datasets. Then, an SNN model has been selected based on its structure involving number of layers and neurons and the synaptic connections. Following this, the SNN should be trained through learning algorithms to achieve desired results for the relevant task. The tools employed in the mentioned stages for SNNs modeling and simulations are usually MATLAB, Python, and C/C++. Scholars and developers can prototype, test, and fine-tune SNN models prior to deployment with the help of these tools. For FPGA implementations, the simulated SNNs are described and translated into HDLs, or alternatively, they are used to automatically generate hardware through high-level synthesis (HLS). This comprehensive approach ensures efficient and successful deployment of SNNs on FPGA platforms, paving the way for accelerated neuromorphic computing applications.

In [41], a cost-efficient and high-speed neural hardware architecture based on an improved version of the digital LIF neuron model in an SNN for a recognition application was presented. In FPGA implementation of the proposed SNN, shift and add modules have been employed instead of costly multipliers. The accuracy rate of the SNN for recognizing patterns was 100%

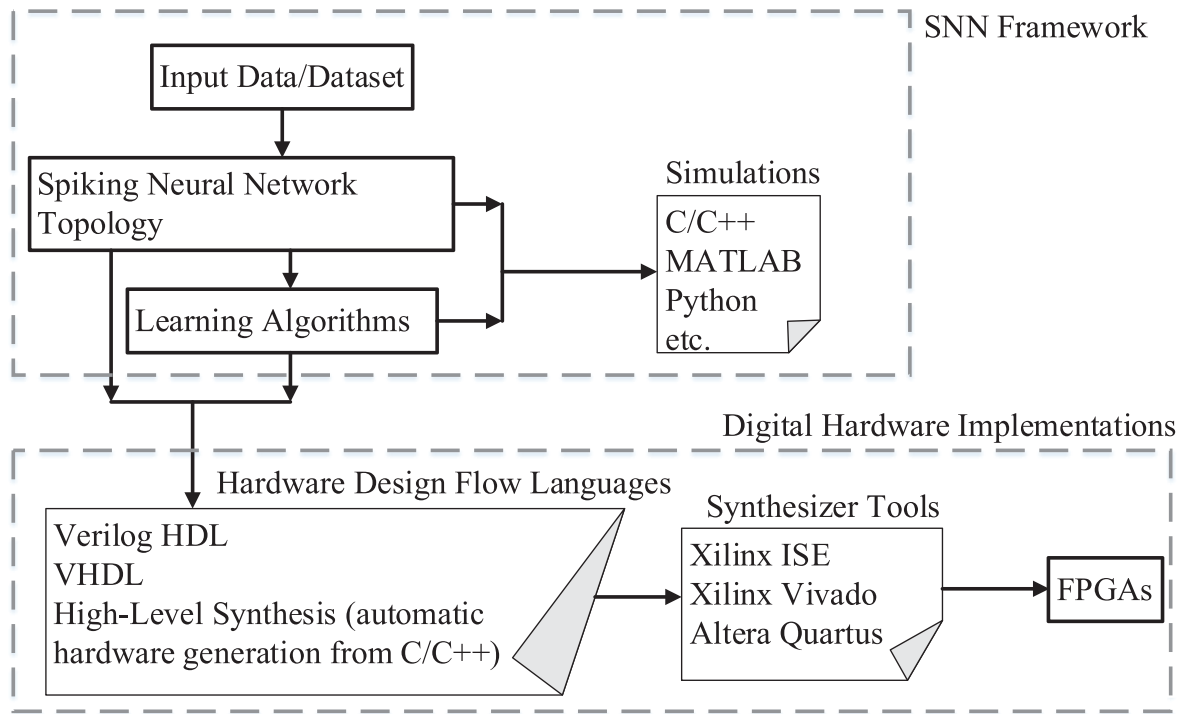


Fig. 7. A diagram illustrating the process of transitioning from software simulations to digital hardware implementations of SNNs, including the tools used at each stage.

without any noisy conditions. Two types of noise called Type I and Type II were introduced to test the robustness of the network. In [63], based on the CORDIC technique, an FPGA implementation of an SNN with online STDP learning was discussed. This method leads to cheap and simple implementations on digital hardware.

In [64], a high-speed and energy-efficient FPGA-based SNN processor with online learning was introduced. Adaptive clock/event-driven computing scheme (ACECS), neighboring PE borrowing technique (NPBT), compressed spike routing technique (CSRT) and reconfigurable PE for inference and learning (RPIAL) are the techniques used in hardware implementation for improvements in power/energy and speed. In [65], a deep SNN was efficiently implemented on an FPGA. The layers were implemented in parallel to obtain low latency and a spiking max-pooling method was employed to alleviate the computation complexity. The convolution operations were accelerated by shift registers and coarsely grained parallel approach. In [66], a fully-connected SNN has been implemented on an FPGA in real-time, which is scalable hardware architecture from small-scale to medium-scale. Although the presented architecture is resource-consuming, the speed of the implemented SNN is high. In [67], an efficient configurable neuromorphic hardware co-processor verified on an FPGA, named Darwin Neural Processing Unit (NPU), on the base of an SNN. Time-multiplexing the physical neural units reduces the use of computational resources. The implemented SNN could be scaled up to possibly millions of neurons by employing the designed NPU as a processing element in a network-on-chip architecture.

In [68], an event-driven digital multiplierless hardware implementation of an SNN with reinforcement learning has been investigated. To validate the digital SNN and the learning capability in hardware, a robot in a closed sensory-motor loop was tested. In [69], the first register-transfer level (RTL) hardware block-based end-to-end architecture for SNNs has been presented. A method for creating spike trains using dynamic quantization from real-valued input samples was described. By increasing the amount of parallelism in the hardware generated, data transfer and logic usage are optimized. The proposed framework was evaluated based on CNN, LeNet-5, AlexNet, and VGG-11. In [70], an SNN has been implemented on an FPGA using 8-bit fast-convergence CORDIC and time division multiplexing approaches to reach the high energy efficiency in hardware, fast on-line learning, and high classification accuracy.

In [71], on-FPGA parallel neuromorphic processor architectures for SNNs have been introduced. With regard to efficient parallelization of membrane potential computation, on-chip storing of synaptic weights, and integration of approximate arithmetic units, the proposed designs satisfactorily make tradeoffs between power/energy consumption, hardware costs, and throughput. The hardware implementations were based on standard booth multipliers and approximate multipliers with fixed-point arithmetic.

In [72], a biologically plausible neural network including two mini-columns structure has been implemented on an FPGA. To obtain accurate response and efficient hardware utilization the CORDIC algorithm and a step-by-step integrator have been employed. In [73], a homeostatic fault tolerance bio-inspired

TABLE XI
COMPARISON OF THE IMPLEMENTED SNNs FROM [41], [63] TO [76]

Ref.	Reg./FFs	LUTs	Speed (MHz)	DSPs	BRAMs	Power / Energy	FPGA Device	Dataset	App.	Acc.	#Neur.
[41]	1023	11339	189.071	0	5	-	X. Virtex-6 ML605	Binary Letters	image classifi.	100%	6
[63]	7088	10376	84.1	0	-	-	X. Spartan-6 XC6SLX75	-	-	-	21
[64]	-	-	100	-	-	- / 5.04 mJ	X. Virtex-7 VC707	MNIST	image classifi.	92.93%	1094
[65]	67278	107273	150	-	264.5	4.6 W / 0.03 J	X. Zynq ZCU102	MNIST	image classifi.	98.94%	78538
[66]	48502	55884	100	408	392	8.5 W / -	X. Virtex-6 XC6VLX240T	-	-	-	1440
[67]	1676	6214	25	32	72	0.84 mW/MHz / -	X. Spartan-6 XC6SLX45	MNIST	image classifi.	93.8%	1794
[68]	8906	19059	148.4	0	-	1.81 W / 2257 μ J	X. Kintex-7 XC7kt160t	-	robotics	-	16
[69]	36000	41000	200	0	-	3.6 W / -	X. Virtex UltraScale+ XC7VU13P	MNIST	image classifi.	99.3%	-
[69]	24000	27000	200	0	-	3.4 W / -		MNIST	image classifi.	99.1%	-
[69]	50000	48000	150	0	-	4.7 W / -		CIFAR-10	image classifi.	80.6%	-
[69]	84000	88000	150	0	-	5 W / -		CIFAR-100	image classifi.	65%	-
[70] ¹	19502	51007	303.4	430	0	- / 5.802 mJ	X. Zynq xc7z035fbg676-2	MNIST	image classifi.	80.8%	-
[70] ²	23320	78053	301.8	430	0	- / 6.646 mJ	X. Zynq xc7z035fbg676-2	MNIST	image classifi.	83.4%	-
[71] ³	58826	97287	120	0	34	- / 1339.83 mJ	X. Virtex-6 ML605	MNIST	image classifi.	89.1%	1591
[71] ⁴	58345	93232	120	0	-	- / 1115.02 mJ	X. Virtex-6 ML605	MNIST	image classifi.	87.7%	1591
[72]	50228	86032	63.386	1112	14	-	X. Virtex-7xq7k410t-21-rf676	-	-	-	150
[73]	8827	3852	30.913	16	-	1.513 W / -	X. Artix-7	-	robotics	-	4
[74]	5088	34646	143	256	-	1.91 W / 139.5 μ J	X. Kintex-7 XC7kt160t	-	robotics	-	16
[75] ⁵	10325	12218	200	64	83.5	0.535 W / 2.23 mJ	X. Kintex-7 XC7K325T	MNIST	image classifi.	97.70%	2842
[75] ⁶	10325	12218	200	64	83.5	0.535 W / -		Fashion-MNIST	image classifi.	85.14%	2842
[75] ⁷	10325	12218	200	64	83.5	0.535 W / 2.30 mJ		MNIST	image classifi.	97.81%	2842
[75] ⁸	10325	12218	200	64	83.5	0.535 W / -		Fashion-MNIST	image classifi.	83.16%	2842
[76]	-	-	75	64	200	1.5 W / -	X. Spartan-6 LX150	MNIST	image classifi.	92%	1794

¹6-bit fast-convergence CORDIC, ²8-bit fast-convergence CORDIC, ³K=32 standard, ⁴K=32 approximate, ^{5,6}LIF neurons, ^{7,8}Izhikevich neurons

architecture has been presented for an SNN. The proposed concept was evaluated by implementing the SNN on an FPGA for a robotic controller application. This work introduces an approach for hardware implementation of the homeostatic self-repair behavior of natural fault tolerant systems. In [74], a digital SNN including reinforcement learning adaptation mechanism for a context-dependent task has been implemented on an FPGA. The introduced digital hardware successfully regenerates animal experiments and computational models on a learning task. The proposed digital circuit fires 10.7 times fewer spikes and speeds up learning by 15 times while consuming 16 times less energy. In [75], an FPGA-based acceleration for SNNs has been discussed. A RISC-V processor and a Xilinx FPGA were employed to implement the SNNs up to simulation of 16384 neurons and 16.8 million synaptic connections. Time-division multiplexing technique, which leads to a scalable accelerator and efficiently speeds up data processing, is used to increase the number of neurons. In [76], Minitaur was introduced as an event-driven SNN accelerator based on an FPGA.

Throughout this paper, we abbreviate registers, application, accuracy, and number of neurons as Reg., App., Acc., and #Neur., respectively. Table XI compares the implemented SNNs on various FPGA devices as presented in [41] and from [63] to [76].

In [77], a biologically inspired auditory framework was implemented on an FPGA. A hierarchical SNN with sparsely connected neurons has been proposed for speech recognition from a set of speech samples spoken by different speakers and mixed with noise. The Bayesian classifier was employed to classify the content of speech based on the prior probability computed from all samples. The FPGA synthesis of the hierarchical SNN presented on-chip speech recognition for the auditory system. The hardware implementation benefits from multiplierless and fully paralleled architecture.

In [78], using two numerical approaches, the Euler method and the third-order Runge-Kutta (RK3) method, an optimization methodology has been proposed for enhancing the performance of a biologically plausible SNN and its FPGA-based digital implementation with parallel architecture. The optimization strategy investigates how biological time constants affect data transfer in the SNN and, with the appropriate selection of the time constants, ameliorates the convergence rate for digit recognition. Compared to the CPU-based software simulation of the SNN, the Euler-based parallel implementation of the SNN on an FPGA achieves a training speedup of about 180 $\times$ and an inference speedup of about 20 $\times$. Although the digital implementation of the SNN with the RK3 method consumes more resources and energy, it outperforms the implementation with the Euler method in terms of training speedup by 2 $\times$, which makes it appropriate for real-time applications requiring online training. In [79], a reconfigurable architecture for neuromorphic computing has been introduced. This architecture is flexible and open-source, which provides rapid software simulation of the neuromorphic systems via C++ and the high-speed hardware emulation by FPGA. The neural layers are initially fully connected. For training the network, the weights of ± 1 are randomly assigned across all the connections. Also, to determine whether a synaptic connection exists between two neurons, synaptic connection probabilities are maintained. The FPGA scalability configuration of the digital design supports up to 259072 neurons and 73317376 synapses.

In [80], a digital implementation of an SNN, trained by backpropagation learning algorithm, has been presented. In [81], a reconfigurable on-chip gradient descent, learning of a two-layered SNN has been implemented. The SNN enabled to successfully approximate three nonlinear functions including sigmoid, 2^x , x^3 with average relative errors of 0.03%, 2.65%, and 0.083%, respectively. To show the digital hardware synthesis, only the approximated sigmoid has been implemented on

an FPGA. For the purpose of efficiency in cost and speed, the expensive multipliers were eliminated by shift and add digital blocks. In [82], FPGA implementation of a single layer SNN has been discussed for a function approximation. The network accurately approximate linear and nonlinear functions like $2x + 3$, x^3 , $\sin x$, and $0.00024414 x (x - 60)(x + 60)$ with average relative errors of 0.0014%, 0.0319%, 0.0833%, and 0.0247%, respectively. The last function was synthesized and implemented on an FPGA.

In [83], a low-cost, low logic, and low memory-consuming resources digital implementation of a spiking extreme learning machine model with triplet-based reward modulated spike-timing-dependent plasticity on an FPGA device has been discussed. In [84], a large-scale architecture for multi-compartment conductance-based neurons has been considered in a randomly connected SNN structure and for efficient scalable real-time implementation on a hardware platform including four FPGAs. In [85], an FPGA implementation for a continuous attractor neural network (CANN) of spiking neurons has been propounded. The CANN is adjusted through various synaptic dynamics to implement brain cognitive functions. The neuromorphic implementation of this structure takes advantage of cognitive computing in real-time. To implement various synaptic dynamics, fast-linear and slow-nonlinear synapses were considered. This capability leads to implementing tasks with slow and nonlinear neural dynamics such as working memory and decision making on an FPGA.

In [86], a digital implementation of spiking multi-kernel convolution architecture is proposed for processing event-driven data obtained from dynamic vision sensor (DVS) chips. To accelerate data processing, a pipeline technique was used. Efficient data arrangement in memories computes the kernel convolution operations in parallel that lead to accelerating memory access and reducing memory cost. The designed FPGA-based processor consumes lower energy and has higher throughput in comparison with other related works. In [87], cost-efficient neural accelerators by reduced area and power consumption based on a recurrent SNN have been explained. A non-Hebbian intrinsic plasticity learning rule as a self-adaptive mechanism was employed to train the network on an FPGA.

In [88], an FPGA-based accelerator has been proposed for online training of an SNN with the spike-timing-dependent plasticity based back-propagation (BP-STDP) training algorithm. In the hardware accelerator, the training time and recognizing time are $41.87\times$ and $56.96\times$ faster compared to the CPU executions.

In [89], a real-time digital platform for implementing large-scale conductance-based spiking neural networks (LaCSNN) is described. The LaCSNN model benefits from both high biological reality and large size of the network. The network is realized on a 3D network-on-chip topology with six FPGAs. Cost-effective conductance-based neurons are put forth at the single neuron level, resulting in an average utilization of 95% fewer memory resources and 100% fewer DSP resources by using piecewise linear approximation and digital multiplierless technique. In [90], an FPGA implementation of an SNN for image classification is presented. An updating algorithm, which

combines the event-driven and traditional discrete time-stepped updating algorithms, has been employed. The designed digital systems can support maximum 16384 neurons and 16.8 million synapses with minimal hardware resources and considerable power consumption.

In [91], in order to make the design of an RRAM-based neuromorphic chip easier and faster, an FPGA-based hardware emulator has been suggested. As RRAM-based crossbars are used in the neural cores of a neuromorphic chip, the emulator supports both static and dynamic variations of these crossbars.

The comparison of the implemented SNNs on various FPGA devices from [77] to [91] has been presented in Table XII.

In [92], a digital hardware architecture based on an FPGA has been developed for an SNN in order to produce locomotion patterns, including walk, trot, and run. The SNN is considered as a central pattern generator (CPG). Grammatical evolution, which is a grammar-based genetic programming technique, has been applied for setting the synaptic weights. The implemented SNN has been successfully evaluated on various types of real robots, called Biped, Quadruped, and Hexapod.

In [93], a neuromorphic digital architecture and its improved version for a large SNN have been recommended in mobile robotic vehicle applications. An FPGA validated both versions of the neuromorphic employing fixed-point arithmetic.

In [94], an energy-efficient convolutional SNN accelerator based on an FPGA, known as Skydiver, has been introduced in order to take advantage of the balance of spatiotemporal workload. The approximate proportional relation construction (APRC) technique was applied to predict the relative workload channel-wisely. In order to increase hardware effectiveness and throughput, the workload balancing method, called channel-balanced workload schedule (CBWS), was proposed.

In [95], the human brain fault-tolerant capability has been mimicked by designing an on-FPGA hardware platform, called Self-rePAIRing spiking Neural NetwoRK (SPANNER). To self-detect and self-repair of the synaptic faults, the SNN should consist of neuron, synapse, and astrocyte. Accordingly, a hardware architecture for a neuron-astrocyte network has been introduced.

In [96], an FPGA acceleration of Spike-by-Spike neural network, which is an SNN based on a generative probabilistic model, has been presented. The main advantages of Spike-by-Spike neural network are its noise tolerance and reduced complexity. A dot-product functional hardware module based on approximate computing has been designed for hybrid custom floating-point and logarithmic computations. The proposed FPGA implementation leads to $20.5\times$ reduction in latency, $8\times$ reduction in weight memory footprint, 12.35% improvement in energy consumption in comparison with the standard floating-point hardware implementation.

In [97], an FPGA implementation of a reconfigurable polychronous SNN with programmable delays, which is highly capable of detecting spatial-temporal patterns, was proposed. A time multiplexing technique was used for the digital implementation. The proposed SNN succeeded to recall over 95% of the spikes for 96% of the stored patterns.

TABLE XII
COMPARISON OF THE IMPLEMENTED SNNs FROM [77] TO [91]

Ref.	Reg./FFs	LUTs	Speed(MHz)	DSPs	BRAMs	Power / Energy	FPGA Device	Dataset	App.	Acc.	#Neur.
[77]	19325	49082	65.03	0	0	1.684 W / 276.83 μ J	A. Cyclone IV	TIMIT	speech recog.	74.35%	66
[78] ¹	50614	45531	100	-	289	- / 4.45 mJ	X. Virtex-7 VC709	MNIST	image classifi.	89.5%	1584
[78] ²	31099	32983	100	-	101	- / 1.73 mJ		MNIST	image classifi.	88.3%	1184
[78] ³	91281	88053	100	-	289	- / 5.19 mJ		MNIST	image classifi.	89.7%	1584
[78] ⁴	67984	54313	100	-	101	- / 1.39 mJ		MNIST	image classifi.	88.6%	1184
[79]	918316	1698995	177.336	-	2684	-	X. Alveo U250	MNIST	image classifi.	98.14%	-
[80]	370	1766	-	-	2	-	X. Virtex-6 ML605	MNIST	image classifi.	93.4%	686
[81]	989	2009	83.209	8	1	-	X. Virtex-6 ML605	-	func. approx.	-	18
[82]	1032	6264	21.207	-	2	-	X. Zynq-7000	-	func. approx.	-	81
[83]	7559	15621	200	8	129	-	X. ZC706	MNIST Fashion-MNIST	image classifi. image classifi.	93% 78.5%	2932 2932
[84]	-	-	-	-	-	-	A. Stratix III EP3SL340	-	-	-	10 ⁶
[85]	35477	21301	-	192	0	0.716 W / -	X. Zynq-7100xc7z100ffg2	-	decision mak. ⁵	-	64
[86]	-	-	100	-	248x36	1.17 mW / 12.08 nJ	X. Zynq ZCU102	-	visual process.	-	64
[87]	14417	37108	100	-	-	107 mW / 520.02 μ J	X. Zynq ZC706	TI46	speech recog.	95.38%	161
[87]	4406	10186	100	-	-	31 mW / 61.07 μ J		TIMIT	speech recog.	71.5%	54
[88]	77580	75996	100	0	-	- / 1.52 mJ	X. Virtex-7 VC707	MNIST	image classifi.	95.3%	1094
[89]	-	-	-	-	-	10.578 W / -	A. Stratix III EP3SL340	-	-	-	10 ⁶
[90]	7309	5381	200	-	40.5	0.477 W / -	X. ZC706	MNIST	image classifi.	97.06%	2842
[91]	352318	339765	-	-	1278	-	X. Virtex-7 XC7VX690T-2	MNIST	image classifi.	96.80%	1562

¹Euler with 800 output neurons, ²Euler with 400 output neurons, ³RK3 with 800 output neurons, ⁴RK3 with 400 output neurons, ⁵decision making & working memory

In [98], a hierarchical SNN for feature extraction in pattern recognition task and its hardware implementation on FPGAs has been discussed. To test the performance of the SNN, five datasets, called flowers (dataset1), fruit taken under indoor fluorescent lamp (dataset2), fruit taken in different time under outdoor light (dataset3), objects taken in a closed room under red, blue, and white illuminant conditions (dataset4), and objects taken under different lights (dataset5), have been considered. The best recognition accuracy of 96.38% was obtained on dataset2. In order to directly generate synthesizable HDL code, Simulink block diagrams from the blocksets of the System Generator Library in MATLAB/Simulink have been designed. In [99], a low-cost high-speed FPGA implementation of an SNN with tempotron supervised learning rules for picture classification application is suggested. The STDP learning technique is the source of the tempotron supervised learning rules, which can efficiently improve interactive accuracy by adjusting the weight of the neural network in a particular direction during the supervised process. A hardware platform for implementing evolving SNNs on an FPGA has been presented in [100]. The synthesized SNN has 1500 configurable logic blocks, which means 63.78% resource utilization.

In [101], hardware architecture to implement FORCE training, which has been applied to a pool of randomly connected spiking neurons, is presented. The single-cycle and pipeline datapath techniques were considered for hardware implementations. Applying the pipeline to the single-cycle microarchitecture considerably enhances the speed. In [102], NeuroFlow as a general-purpose and scalable SNN simulation platform for high-performance computing through FPGAs is investigated. By employing an FPGA, NeuroFlow provides 33.6 times higher speed than a conventional processor, or on average 2.83 times higher speed than GPU-based platforms. With the features of flexibility and high throughput, NeuroFlow presents a suitable platform for simulating large-scale neural networks. In [103], a scalable and parallel architecture of SNNs for versatile applications (SNAVA) is implemented on an FPGA. The performance of SNAVA framework is 129 $\times$ and 4 $\times$ faster than the

CPU and GPU executions, respectively. Special-purpose processing elements (PEs) and a 3-stage pipelined sequencer are employed to speed up the computations with optimal utilized resources. Also, a distributed memory has been presented for implementing the neural computing in BRAMs to optimize power consumption. In [104], a hippocampal spiking neural network (HSNN) is implemented in real-time on a scalable FPGA-based digital platform. This implementation provides a revolutionary network-on-chip for randomly connected SNNs.

In [105], a scalable and traffic-aware adaptive NoC router is presented. It is a component of the EMBRACE (EMulating Biologically-inspiRed ArChitectures in hardware). EMBRACE is an embedded mixed-signal SNN architecture. The behavior of the adaptive NoC router is verified on an FPGA.

A 3D network on chip architecture for implementing an SNN on an FPGA was presented in [106]. The SNN identifies pneumonia such as COVID-19 diagnosis based on chest X-ray benchmark images. The topology of the SNN is convolutional. The SNN takes advantage of a fault-tolerant spike routing scheme and event-driven processing that lead to energy/power efficiency with low latency. The neuromorphic framework keeps the recognition accuracy rate even when up to 30% of the communication channels in the NoC are faulty. However, the latency increased due to 30% faults detected in the system.

Table XIII compares the implemented SNNs on various FPGA devices from [92] to [106].

In [107], an SNN inference engine for accelerating spiking convolutional neural network (SCNN), called SIES, implemented on an FPGA. SIES expedites the inference of the SCNN. The peak performance of SIES was 1.5625 TOPS. The MNIST, SVHN, and CIFAR-10 datasets were employed to evaluate the performance of the SCNN. The error rates of 0.84%, 4.66%, 18.20% were obtained for three different networks on MNIST, SVHN, CIFAR-10, and an 8.54% error rate was achieved for VGG-16 network on CIFAR-10.

In [108], a high-level metric, called the synaptic activity ratio (SAR), is discussed in order to reduce the energy consumption

TABLE XIII
COMPARISON OF THE IMPLEMENTED SNNs FROM [92] TO [106]

Ref.	Reg./FFs	LUTs	Speed (MHz)	DSPs	BRAMs	Power / Energy	FPGA Device	Dataset	App.	Acc.	#Neur.
[92] ¹	383	626	-	0	-	1 mW / -	X. Spartan-6 XC6SLX45	-	robotics	-	6
[92] ²	492	857	-	0	-	1 mW / -		-		-	8
[92] ³	494	857	-	0	-	1 mW / -		-		-	8
[92] ⁴	496	988	-	0	-	1 mW / -		-		-	8
[92] ⁵	492	836	-	0	-	1 mW / -		-		-	12
[92] ⁶	492	849	-	0	-	1 mW / -		-		-	12
[92] ⁷	488	938	-	0	-	1 mW / -		-		-	12
[93]	-	-	40	-	-	-	X. Virtex-II XC2V1000FG256-4	-	robotics	-	>1000
[94]	20544	45986	200	0	262	0.96 W / 42.4 μ J	X. XC7Z045	MNIST	image classifi.	98.5%	-
[95]	10383	9865	200	45	-	-	X. Virtex-7 VC707	-	fault tolerance	-	2
[96] ⁸	56036	39514	200	82	180	2.420 W / -	X. Zynq-7020	MNIST	image classifi.	98.98%	-
[96] ⁹	58759	42021	200	82	156	2.369 W / -		MNIST	image classifi.	98.97%	-
[96] ¹⁰	57862	41060	200	82	148	2.324 W / -		MNIST	image classifi.	98.84%	-
[97]	58341	134665	66	-	402	-	X. Virtex-6 XC6VLX240T	-	pattern recall	-	4096
[98]	-	-	-	-	-	-	-	-	image classifi.	96.38%	-
[99]	5487	13862	178	144	-	-	X. Virtex-7	-	color classifi.	96%	51
[100]	-	-	54.4	-	-	-	X. Spartan-II xc2s200	-	frequency discrim.	-	30
[101]	10619	8621	170.7	42	132.5	-	X. Artix-7 XC7A200T	-	pattern generation	-	510
[102]	-	-	145	-	-	30-40 W / -	A. Stratix V 5SGSD8	-	-	-	589824
[103]	97824	148774	-	100	213	0.625 W / -	X. Kintex-7 XC7K325T	-	-	-	12800
[104]	-	28558	-	-	-	-	A. Stratix III EP3SE260	-	hippo. oscillation ¹¹	-	10000
[105]	-	-1200	-180	-	-	1.72 mW / -	A. Stratix II	-	-	-	-
[106]	37098	27288	100	0	0	0.007 mW / -	X. Zynq ZCU102	X-ray	image classifi.	88.43%	-

¹biped walk, ²quadruped walk, ³quadruped jog, ⁴quadruped run, ⁵hexapod walk, ⁶hexapod jog, ⁷hexapod run, ⁸standard floating-point computation,

⁹hybrid custom floating-point approximation, ¹⁰hybrid logarithmic approximation, ¹¹hippocampal oscillation

in hardware implementation of SNNs. Moreover, an estimation framework is proposed to compare SNNs and formal neural networks (FNNs) accelerators under different levels of parallelism for the same datasets. The framework evaluates the logic resource utilization, power/energy consumption, and execution time under two levels of parallelism (highly sequential and highly parallel) by four hardware neural network accelerators architectures on an FPGA. The datasets employed were MNIST, GTSRB, CIFAR-10, Spoken Digits, and RadioML. The best accuracy rates were 97.3%, 98.9%, and 99.1% for different SAR values on the MNIST dataset.

In [109], an FPGA-based layered tile architecture (LTA) is introduced which uses a two-level sharing mechanism for addressing the resource challenge of hardware SNNs. It can provide a balance between computational complexity and hardware resource cost. The proposed LTA is used to minimize the resource cost while requiring low DSP resources to implement a large-scale SNN.

In [110], an SNN composed of three types of neurons for real-time speech recognition is recommended. To prove the real-time recognition, the SNN employed for Spanish vowel phonemes recognition has been described by VHDL and implemented on an FPGA. To transfer the information between the neurons an address event representation (AER) is utilized in the implementation. The AER benefits from the event-driven computing that leads to efficient and fast computations.

In [111], a hierarchical modular architecture for implementing an SNN on an FPGA was investigated. In order to facilitate spike communication across neural tiles, a fixed spike transfer latency ring architecture interconnect has been presented. The spike transfer latency continues the integrity of information and eliminates the distortions on the data.

In [112], a biomimetic CORDIC-based FPGA implementation of an SNN consisting of synapses and synaptic noise is studied. The adjustable and scalable architecture is a suitable platform for studying neurological diseases. In [113], a synchronous

method, named SyncNN, with more hardware-friendly and computational performance compared to conventional asynchronous techniques is introduced. The SyncNN framework is designed and implemented for the acceleration of the SNNs on an FPGA. Memory access and computational efficiency are ameliorated using the SNN quantization methods. The bit precision for the weights of the SNN was 16-bit, 8-bit, and 4-bit fixed-point. SyncNN employed LeNet, Network in Network, and VGG on various datasets such as MNIST, SVHN, and CIFAR-10. The quantized SNN with 4-bit floating-point precision directs to the highest accuracy rate of 99.6% on the MNIST dataset for the LeNet.

In [114], an energy-efficient SNN-based FPGA implementation using approximate adders is presented, which utilizes approximate arithmetic units and variable precision weights. To indicate the minimized bit-width required, the variable precision method is employed to express the weights of the SNN. This approach avoids any loss in approximate computations. The accuracy rates of the four SNN models, including 1994, 3194, 3194, and 4394 neurons, with different configurations on the MNIST-3, MNIST-4, FASHION-4, and FASHION-5, were 98.32%, 98.44%, 86.76%, and 87.36%. Additionally, energy/img values for the reference networks on the MNIST-3, MNIST-4, FASHION-4, and FASHION-5 databases are 150.47 μ J, 373.41 μ J, 373.41 μ J, and 596.33 μ J.

In [115], an event management algorithm, called the structured heap queue (SHQ), is introduced as a pipelined digital hardware data structure for the efficient FPGA implementation of large-scale event-driven SNNs. The SNN was tested on image segmentation task by employing a 406×158 pixel image. Each event is processed every 7 clock cycles without considering the network size. In [116], high-performance architecture of an SNN, called streaming spiking neural network (S2NN), is explored to improve data precision and distribution of configuration data stored in the main memory. The S2NN was implemented by high-level synthesis design flow on a hybrid

TABLE XIV
COMPARISON OF THE IMPLEMENTED SNNs FROM [107] TO [121]

Ref.	Reg./FFs	LUTs	Speed (MHz)	DSPs	BRAMs	Power / Energy	FPGA Device	Dataset	App.	Acc.	#Neur.
[107]	421230	302797	200	0	192	-	X. XCVU440	MNIST	image classifi.	-	4000
[108]	5200	19300	-	0	0.04 Mb	248 mW / 0.02 mJ	X. Zedboard Z-7020	MNIST	image classifi.	99.1%	886
[109]	-	-	200	-	-	43.7 mW / -	X. Zynq-7000 XC7Z020-CLG484	-	XOR problem	-	1728
[110]	6811	4561	-	-	-	- / -	A. Cyclone IV E EP4CE115F29C7	words ¹	speech recog.	-	-
[111]	1175	-	200	0	1	- / -	X. Virtex-6 XC6VLX240T	-	data distortion	-	448
[112] ²	2360	5551	100	28	-	- / -	X. Kintex-7 xc7k325tffg900	-	neural diseases	-	500
[112] ³	71608	167346	100	840	-	- / -	X. Kintex-7 xc7k325tffg900	-	neural diseases	-	15000
[113]	-	-	200	-	-	24.5 W / -	X. ZCU102	MNIST	image classifi.	99.6%	-
[114]	13050	4962	250	-	516	442.32 mW / *	A. Arria-10 10AS066N3F40E2SG	MNIST-4	image classifi.	98.44%	3194
[115]	3368	4673	100	-	130	-	X. XC5VSX50T	Image**	image segment.	-	65536
[116] ⁴	15000	28000	150	-	-	0.7 W / 2300 mJ	X. Zynq 7100	-	data streaming	-	22500
[117]	12694	43975	100	-	-	161 mW / -	X. Zynq ZC-706	TI46	speech recog.	91.53%	135
[117]	12717	45785		-	-	195 mW / -				93.46%	135
[117]	19841	57581		-	-	237 mW / -				95%	135
[117]	19844	57788		-	-	229 mW / -				93.84%	135
[118]	255	549	-	-	-	- / -	X. Spartan-6 XC6SLX45	-	robotics	-	12
[119]	25430	29130	71.4	280	20	- / -	X. Artix-7 XC7A200T	-	-	-	4096
[120]	-	-	-	-	-	10 W / 14 nJ	X. Spartan-6 XC6SLX45T	-	-	-	262× 10 ³
[121] ⁵	12357	13761	200	0	277.5	0.699 W / 0.04 mJ	X. Kintex-7 XC7K325T	MNIST	image classifi.	98.35%	1754
	12357	13761	200	0	277.5	0.699 W / 0.10 mJ		F-MNIST	image classifi.	89.01%	1754
	12357	13761	200	0	277.5	0.699 W / 0.19 mJ		SVHN	image classifi.	70.80%	1754
[121] ⁶	138658	80172	100	0	245.5	0.982 W / 1.19 mJ		MNIST	image classifi.	99.10%	-
	138658	80172	100	0	245.5			F-MNIST	image classifi.	90.29%	-
	138658	80172	100	0	245.5			SVHN	image classifi.	82.15%	-

¹Spanish phonemes & words, ²one core, ³30 cores, ⁴150 × 150 neurons, ⁵spiking MLP topology, ⁶spiking convolutional topology, *373.41 μ J, **406 × 158 pixel image

CPU-FPGA hardware platform. The digital implementations of the S2NN have been evaluated by different numbers of neurons. The scalability of the network is up to 250 K neurons and 125 M synapses.

In [117], FPGA implementations of bio-plausible supervised and unsupervised STDP learning algorithms for training of a recurrent spiking neural network model, known as liquid state machine (LSM), are explained. The FPGA LSM neural accelerators benefit from different leaning schemes, named fixed + baseline, unsupervised STDP + baseline, unsupervised STDP + CaL-S²TDP, and unsupervised STDP + CaL-S²TDP & CaS-S²TDP. In [118], digital hardware design of a central pattern generator (CPG) embedded in a hexapod robot locomotion system is presented. The CPG performs as a network of spiking neurons to generate rhythmic signals for the walking, jogging, and running of the hexapod robot. To take advantage of fine-grained parallelism in the FPGA, the implementation was considered fully-paralleled.

In [119], a randomly connected SNN was implemented on an FPGA. The number of spikes at each synapse in the sparse network is counted using an approximate two-level counter. Also, a 7-stage pipeline technique was employed to improve the throughput. The approximate and pipeline approaches provide scalable, high-performance, and cost-optimized hardware architecture for the execution of large-scale SNNs. In [120], a hierarchical address event routing (HiAER) architecture for running large-scale SNNs is elucidated. The system is implemented on five FPGAs and four custom ICs.

In [121], an FPGA-based neuromorphic hardware acceleration system (NHAS) to support various SNNs topologies is offered. By employing the optimization technique of extended prediction correction (EPC), an LIF spiking neuron is designed in order to reduce hardware resources and computational complexity. Both optimized spiking neuron and the neuromorphic

hardware acceleration system have been designed by parallel and pipeline techniques to run SNNs efficiently.

The implemented SNNs on various FPGA devices from [107] to [121] have been compared in Table XIV.

In [122], an FPGA-based accelerator of SNN and CNN combination for classifying images was explored. The network is event-driven and recurrent. In [123], a high-level synthesis implementation of an SNN with semi-supervised learning technique for pattern recognition was suggested. A dimensionality reduction based on a principal component analysis (PCA) method to reduce the hardware resources and speed up the process is discussed. The hardware optimization techniques like array partition, dataflow, and pipeline were utilized to diminish latency and accelerate computing in the synthesized SNN.

In [124], a fault-tolerant neuromorphic digital hardware for context-dependent learning tasks is introduced. A fault-tolerant spike routing technique was presented to show that the learning of the neuromorphic system cannot be interrupted by diverse faults, and the context-dependent learning would be processed without any disturbances. A monkey navigation model was proposed for decision-making applications in the context-dependent learning. Additionally, utilizing the suggested routing strategy, it was demonstrated that the system's average latency and maximum throughput were greatly enhanced in comparison with earlier routing schemes.

In [125], a liquid state machine, which is a type of SNNs, has been implemented on an FPGA for digits speech recognition. The weights of the network were randomly selected. The hardware architecture proposed for the implementation is based on serial arithmetic processing.

In [126], a hippocampus-inspired recurrent spiking neural network has been implemented based on the CORDIC algorithm. The spiking neurons were implemented by the CORDIC algorithm that enhances the performance of the large-scale

TABLE XV
COMPARISON OF THE IMPLEMENTED SNNs FROM [122] TO [136]

Ref.	Reg./FFs	LUTs	Speed(MHz)	DSPs	BRAMs	Power / Energy	FPGA Device	Dataset	App.	Acc.	#Neur.
[122]	-	-	250	147	71	2.039 W / 0.050 nJ/SO	X. Zynq 7020	MNIST	image classifi.	99.3%	2954
[123]	73852	31973	-	192	116	- / -	X. Zynq ZC702	MNIST	image classifi.	95.7%	2010
[124]	~(10%)	~(62%)	-	0	0	- / -	A. EP3E340	-	decision-making	100%	16
[125]	-	1384	100	0	56	- / -	X. Virtex-4 ML401	-	speech recog.	-	200
[126]	11546	10642	146.03	0	-	- / -	A. Stratix III EP3SE260	-	hippo. oscillation	-	64000
[127]	8713	23573	200	0	431.5	0.782 W / 16.83 mJ	X. ZCU102	MNIST	image classifi.	90.53%	3098
[128]	2752	0	54.99	0	-	329.43 mW / -	A. Stratix III EP3SL340	-	pattern generation	-	4
[129]	-	-	100	-	-	- / -	X. Virtex-7 VC709	MNIST	image classifi.	89.58%	-
[130]	1003	1335	125	-	-	142 mW / -	X. Zynq XC7Z010-1CLGC400C	MNIST	image classifi.	90.70%	270
[131]	4427	5535	-	-	28	- / -	X. Kintex UltraScale KU115	-	pattern recog.	-	178
[132]	3427	4502	124	-	19	- / -	X. Spartan-6 XC6SLX150	-	spinal cord inj.	-	7
[133]	115144	140092	125	288	-	- / -	A. Stratix IV EP4SGX530KH40C2	-	Parkinson states	-	512
[134]	33707	25065	-	30	-	0.082 W / -	X. Artix-7	-	safety & security	-	-
[135] ¹	-	-	-	-	-	- / -	-	MNIST	image classifi.	95%	1584
[135] ²	-	-	-	-	-	- / -	-	MNIST		92%	1794
[135] ³	-	-	-	-	-	- / -	-	MNIST		97%	13594
[136]	8505	10052	250	32	131	0.938 W / 0.35 mJ	X. Zynq-7045	MNIST	image classifi.	95.10%	1040

¹SCWN, ²SDBN, ³SCNN

hippocampal network. There was 20% sparsity in the synapses connecting the neurons. Torus-based NoC design for hardware architecture was employed to make the neuromorphic digital systems highly scalable.

In [127], a cost-efficient and high-performance SNN processor for a pattern recognition task was presented. A synapse delay management and 2-bit weights implementation techniques were employed to reduce memory usage. To reduce the delay, a low-cost spike encoder and VFA (Vote-For-All) decoder were employed.

In [128], the performance and efficiency of a respiratory central pattern generator (CPG) SNN including four conductance-based neurons on an FPGA were investigated. The arithmetic computations for each neuron were implemented by pipeline technique that requires several multipliers. A modified version of the CORDIC algorithm and multiplierless approximation technique were employed to reduce the computational complexity of the network and speed up the hardware implementation. In [129], FPGA implementations of an SNN with various pruning techniques were employed. The pruning approach is used to reduce the size and complexity of the SNN. To implement the SNN, a parallel digital implementation approach is provided. These strategies aim to showcase notable reductions in memory usage and enhance energy efficiency. In [130], an SNN implementation using bit-serial hardware architecture has been proposed, named spiking hybrid network (SHiNe). The hardware code for integrating each neuron into the neural network is generated automatically by a software program. The SHiNe model merges the characteristics of artificial and spiking neural networks. The duty-cycle encoding technique is applied to the impulses sent between neurons. This enables efficient routing and minimizes hardware utilization. Also, hardware resources have been more diminished by restricting the number connections from the neurons between the layers, which is known as pruning.

In [131], a spatio-temporal memory (STM) paradigm, in which an SNN stores information in a hierarchical manner, and its hardware implementation for pattern recognition are discussed. Tempotron learning is used to update the inter-layer synaptic weights throughout the learning phase, whilst STDP

is used to change the intra-layer synaptic plasticity. Reducing the neural assembly size in each layer leads to a cost-efficient FPGA implementation. In [132], an FPGA-based SNN has been presented as a bio-inspired controller to answer the demand for an adaptive ventilatory control system in spinal cord injury applications. A rat computational model was used to emulate open- and closed-loop paradigms. The shift operations and pipeline are the hardware optimization techniques employed for cost- and time-efficient implementation of the open-loop SNN.

In [133], a large-scale conductance-based SNN formed by sparse synaptic connections has been simulated and translated to a hardware platform. The proposed SNN explores the mechanism of Parkinson's states. To accelerate the speed and reduce the utilized resources cost look-up-table (LUT) is employed to approximate the nonlinear terms and the multiplications with constant parameters are approximated by shifts and adders/subtraction. The FPGA speeds up the SNN running by 75× faster than the execution on an Intel Core i7-8700K CPU.

In [134], a spiking astrocyte neural network (SANN), which considers the astrocyte contribution, for hardware Trojan detection was addressed. The learning is a combination of the STDP and Bienenstock-Cooper-Munro learning methods, named STDP-BCM. The proposed FPGA implementation accelerates the Trojan detection process by 10⁴ times faster than the software simulation of the SANN that shows the effectiveness of the method for real-time cryptographic applications.

In [135], a reconfigurable digital accelerator, named CyNAPSE, is created to accelerate SNNs. The implemented SNNs are spiking competitive winner-take-all network (SCWN), spiking deep belief network (SDBN), and spiking convolutional neural network (SCNN). The adaptive memory management technique led to reduction in the power consumption by up to 22% in comparison with the traditional methods. According to runtime leakage mitigation strategies, CyNAPSE hardware modules can reduce leakage energy consumption by up to 99.92% and at least 14%.

In [136], a low-cost high-speed event-driven FPGA-based SNN accelerator has been implemented for object recognition tasks. The PEs in computing blocks are substantially pipelined in order to enhance the performance of the neuromorphic hardware

core. The proposed learning algorithms combined the STDP rule with the self-organizing map (SOM) and reinforcement learning schemes. The object recognition accuracy of the SNN accelerator with the SOM-STDP & R-STDP learning rule on the MNIST, ETH-80, ORL-10, Yale-10, N-MNIST, Poker-DVS, and Posture-DVS datasets was 95.10%, 80.89%, 97.50%, 94.94%, 82.32%, 100%, and 97.93%, respectively. Similarly, the accuracy rates of the accelerator with the R-SOM-STDP learning rule were 91.92%, 66.67%, 100%, 94.55%, 65.15%, 100%, and 94.91% on the mentioned databases, respectively.

Table XV compares the outcomes of the discussed SNNs on various FPGA devices from [122] to [136].

In [137], an efficient accelerator for a convolutional SNN (CSNN) with a high level of spike sparsity for event-driven processing has been proposed. The spiking neurons were trained by a conversion approach in the CSNN. The weights from the CNN are converted by a tool, named SNN-Toolbox, and then quantized to 8 bits and 16 bits. A neuron multiplexing scheme was presented to alleviate the memory utilization. To improve the performance of memory interlacing, which is a memory distribution approach, was considered. Memory interlacing makes it possible to distribute memory units in a highly-parallel fine-grained manner. The spike events are compressed into queues in order to take advantage of the high level of sparsity. Even if there is still potential to utilize more PEs, the queue-based processing makes sure that PEs are dedicated as much as required. The latency for both quantization methods was 0.04 ms.

In [138], FPGA implementations of an SNN and its equivalent CNN using the high-level synthesis technique were discussed.

In [139], a configurable spiking convolution architecture was introduced. Two event-driven reconfigurable SCNNs using various coding approaches were designed and implemented to validate the efficiency of the proposed architecture. The presented SCNNs classify the handwritten grey-scale images. The first one employs a rate coding scheme and the second one uses temporal coding scheme. The FPGA processing time for each of the mentioned coding approach is 6.61 ms and 0.10 ms, respectively.

In [140], an SNN-LEGION (Locally Excitatory, Globally Inhibitory Oscillator Network) configuration as a method for normalizing synaptic potentials and performing simultaneous binary image segmentation of different objects while ensuring minimal processing complexity was described. The neuromorphic hardware architecture was implemented by 16-bit arithmetic fixed-point with an array of 6×6 PEs. The proposed SNN-LEGION algorithm and hardware architecture when used together may make it possible to create and progress useful video segmentation tasks.

In [141], an event-driven FPGA-based multiconvolution neuromorphic architecture has been presented. By this system, up to 64 convolution modules with various kernel sizes can be computed. To reduce memory access and latency, the data are processed row-by-row in memory. The implemented convolutions with the kernel size of 1×1 resulted in $1.3 \mu\text{s}$ latency and with the kernel size of 7×7 resulted in $9.01 \mu\text{s}$ latency.

In [142], Cerebron, an event-driven reconfigurable accelerator architecture to implement SNNs for image classification and segmentation, was introduced. The impressive specification of

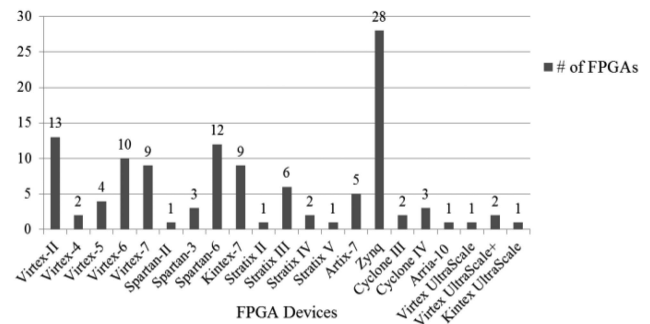


Fig. 8. Distribution of FPGA board types across research papers.

the Cerebron architecture is the exploitation of the spatiotemporal sparsity of the SNNs to reduce time and energy consumption and improve the throughput. To achieve memory efficiency, a reconfigurable compute engine provides data reuse and parallel data access in processing of convolutions. Moreover, the latency is decreased by the suggestion of an online channelwise workload scheduling mechanism. Three deep neural networks were converted to the SNNs using open-source SNNtoolbox. The spiking ConvNet, spiking MobileNet, and spiking SegNet were tested on MNIST, CIFAR-10, and MLND-Capstone databases, respectively. The obtained throughputs for the networks were 40.1 GSOp/s, 44.2 GSOp/s, and 45 GSOp/s.

In [143], a real-time digital implementation of a spiking winner-take-all (SWTA) network was suggested for auditory attention task. Pipelining, resource sharing, and data storage are the optimization

techniques employed to improve the real-time performance of the implementation.

In [144], an approach to accelerate a fully-connected SNN on an FPGA was suggested. This technique, which is known as axonal delay, makes use of a physiological delay that has an impact on the spike's ability to spread through the axon in the neural cell. The hardware system configuration by the proposed method enables real-time imitation of the SNN with 0.1 ms resolution and 3 ms axonal delay. To do this, the synaptic weights are sent via the 4 AXI high-performance ports and saved in the off-chip DDR rather than the internal BRAMs because the capacity of the BRAMs is insufficient to store the weights.

In [145], an event-driven neuromorphic system based on a 3-D butterfly fat tree (BFT) architecture, called CerebelluMorphic, has been discussed. The proposed architecture is used for simulating a large-scale SNN on digital hardware. The digital architecture supports supervised motor learning in cerebellum. Six FPGAs were used to implement the CerebelluMorphic architecture.

The results of the implemented SNNs from [137] to [145] have been compared in Table XVI.

According to the Tables provided for synthesis results, from Tables I to XVI, we have drawn a bar chart and presented it in Fig. 8 to show the distribution of various types of FPGA devices employed in the reviewed papers. As shown in the bar chart in Fig. 8, the Zynq FPGA family is the one that has been most widely used. However, selecting an FPGA device for synthesis

TABLE XVI
COMPARISON OF THE IMPLEMENTED SNNs FROM [137] TO [145]

Ref.	Reg./FFs	LUTs	Speed (MHz)	DSPs	BRAMs	Power / Energy	FPGA Device	Dataset	App.	Acc.	#Neur.
[137] ¹	12000	19000	333	32	2.1 Mb	2.1 W / -	X. Zynq XCZU7EV	MNIST	image classifi.	98.3%	-
[137] ²	21000	33000	333	64	3.9 Mb	2.9 W / -	X. Zynq XCZU7EV	MNIST	image classifi.	98.2%	-
[138]	-	-	100	-	-	0.654 W / 1.52 mJ	X. ZCU102	MNIST	image classifi.	97.7%	1194
[138]	-	-	100	-	-	0.669 W / 14.12 mJ	X. ZCU102	MNIST	image classifi.	99.05%	-
[139] ³	4246	5755	100	-	22	70 mW / 0.46 mJ	X. Kintex-7	handwritten img ⁵	image classifi.	99%	-
[139] ⁴	4246	5755	100	-	47	74 mW / 0.007 mJ	X. Kintex-7	handwritten img ⁵	image classifi.	95.4%	-
[140]	213	1224	50	-	-	- / -	X. Spartan-3 XC3S5000	-	image seg. ⁶	-	31
[141]	172607	247472	100	-	710	1.608 W / -	X. Zynq-7100 MMP	Poker-DVS	image process.	-	-
[142]	70544	85926	200	0	283	1.4 W / 0.04 mJ	X. Zynq XC7Z100	MNIST	image classifi.	99.40%	-
[142]				0		1.4 W / 14.88 mJ		CIFAR-10	image classifi.	91.90%	-
[142]				0		1.4 W / 1.12 mJ		MLND-Capstone	image classifi.	97.30%	-
[143]	-	134000	65.32	92	-	- / -	X. Artix-7 XC7A200T	-	auditory att. ⁷	-	56000
[144]	35158	24480	150	140	130.5	- / -	X. Virtex-6 XC7Z020	-	neuropros. ⁸	-	3098
[145]	-	-	-	-	-	- / -	A. Stratix III EP3SL340	-	motor control	-	3.5×10 ⁶

¹8 bits, ²16 bits, ³rate coding scheme, ⁴temporal coding scheme, ⁵handwritten grey-scale images, ⁶image segmentation, ⁷auditory attention, ⁸neuroprosthesis development

and implementation of digital hardware depends on different factors such as accessibility, availability, hardware resources (including total available registers, flip-flops, LUTs, BRAMs, and DSPs), and even applications.

In hardware implementations of SNNs, the number of implemented neurons on hardware is a very important parameter that should be considered for analyzing resource utilization, power/energy consumption, and speed. However, it is crucial to consider other factors such as spiking neuron models, learning algorithms, structure of the SNNs, and on-chip/off-chip learning approaches. Table XVII provides these details extracted from the papers discussed in this survey (Neur., Learn., Alg., and NN. are shortened formats of neuron, learning, algorithm, and neural network). On-chip (or on-line) learning and off-chip (or off-line) learning are two distinct approaches in hardware implementations of neural networks that discuss where the learning process occurs and how the weights of the model are updated. In the on-chip learning process, the entire learning/training process is done directly on the hardware, in the case of FPGAs, and the weights are updated (adjusted and optimized) continuously till the output is produced on the hardware platform. This technique is suitable for real-time processing and low-latency updates. In the off-chip learning process, the learning/training process occurs externally (out of the hardware), usually from software simulations on computational platforms like CPUs or GPUs. Once training is complete, the trained weights from software simulations are stored and then the pre-trained weights are transferred to FPGAs. In this technique, learning does not happen on FPGAs and the hardware only runs the learned neural networks with fixed weights.

Table XVIII provides an overview of the pros and cons of the selected papers, highlighting key strengths and limitations in their methodologies and findings. These papers were chosen not only for their citations but also based on their perceived importance, novel approaches, relevance to current challenges, and potential impact on future developments in FPGA-based SNNs.

According to Table XVIII, significant advancements in re-configurable neuromorphic computing have focused on low-power/energy consumption, high-speed processing, and noise-resilient systems. Key insights include the importance of noise robustness (e.g., adaptive clock/event-driven computing and

TABLE XVII
OVERVIEW OF SPIKING NEURON MODELS, LEARNING ALGORITHMS, NETWORK TYPES, AND ON-CHIP/OFF-CHIP LEARNING

Ref.	Neur. Model	Learn. Alg.	NN. Type	On/Off-Chip	Ref.	Neur. Model	Learn. Alg.	NN. Type	on/off-chip
[41]	LIF	STDP	SNN	off-chip	[104]	Izhi	-	HSN	-
[63]	Izhi	STDP	SNN	on-chip	[105]	LIF	-	SNN	-
[64]	LIF	STDP	SNN	on-chip	[106]	LIF	BP ¹³	CSNN	off-chip
[65]	IF	BP ¹	DSNN	-	[107]	IF	-	SCNN	-
[66]	Izhi	-	SNN	-	[108]	IF	BP ¹⁴	SNN	-
[67]	LIF	-	SDBN	off-chip	[109]	LIF	-	SNN	-
[68]	LIF	STDP	SNN	on-chip	[110]	SN ¹⁵	-	SNN	-
[69]	-	-	CNN	-	[111]	LIF	-	SNN	-
[70]	LIF	STDP	SNN	on-chip	[112]	HH	-	SNN	-
[71]	LIF	STDP	SNN	on-chip	[113]	IF	CoS ¹⁶	SCNN	-
[72]	HH	-	SNN	-	[114]	LIF	BP ¹	SDN	-
[73]	LIF	-	SNN	-	[115]	LIF	-	SNN	-
[74]	LIF	STDP	SNN	on-chip	[116]	Izhi	-	S2NN	-
[75]	LIF	Mig ²	SNN	-	[117]	LIF	STDP	RSNN	on-chip
[76]	LIF	-	SDBN	off-chip	[118]	IF	-	SNN	-
[77]	LIF	-	HSNN	-	[119]	HH	-	SNN	-
[78]	LIF	STDP	SNN	on-chip	[120]	IF	-	SNN	-
[79]	LIF	-	SNN	-	[121]	LIF	A2S ¹⁷	SNN	off-chip
[80]	Wil. ³	BP ¹	SNN	on-chip	[122]	LIF	A2S ¹⁷	SCNN	off-chip
[81]	Izhi	GD ⁴	SNN	on-chip	[123]	LIF	STDP	SNN	on-chip
[82]	LIF	GD ⁴	SNN	off-chip	[124]	LIF	STDP	SNN	on-chip
[83]	LIF	STDP	SELM	on-chip	[125]	LIF	-	RSNN	-
[84]	HH	STDP	SNN	-	[126]	Izhi	-	RSNN	-
[85]	LIF	-	CANN	-	[127]	LIF	STDP	SNN	-
[86]	LIF	-	SCNN	-	[128]	HH	-	SNN	-
[87]	IF	IP ⁵	RSNN	on-chip	[129]	LIF	STDP	SNN	on-chip
[88]	IF	STDP	SNN	on-chip	[130]	-	BP ¹	SNN ¹⁸	-
[89]	HH	-	SNN	-	[131]	LIF	STDP	STM	off-chip
[90]	LIF	HyUp ⁶	SNN	off-chip	[132]	LIF	STDP	SNN	-
[91]	IF	-	SNN	-	[133]	HH	-	SNN	-
[92]	LIF	GE ⁷	SNN	-	[134]	LIF	STDP	SNN ¹⁹	-
[93]	LIF	-	SNN	-	[135]	LIF	STDP	SNN ²⁰	-
[94]	-	-	CSNN	-	[136]	LIF	STDP	SNN	on-chip
[95]	LIF	-	SNN ⁸	-	[137]	IF	Con ²¹	CSNN	off-chip
[96]	-	MGD ⁹	CSNN	-	[138]	IF	BP ¹	SNN	-
[97]	-	-	PSNN	-	[139]	LIF	-	SCNN	-
[98]	IF	STDP	HSNN	-	[140]	LIF	-	SNN	-
[99]	LIF	Tem ¹⁰	SNN	on-chip	[141]	LIF	-	SCNN	-
[100]	IF	Heb ¹¹	SNN	on-chip	[142]	IF	Con ²¹	SCNN	-
[101]	Izhi	FOR ¹²	SNN	-	[143]	Izhi	-	SNN ²²	-
[102]	Izhi	STDP	SNN	-	[144]	Izhi	-	SNN	-
[103]	LIF	STDP	SNN	-	[145]	LIF	STDP	SNN	-

***Backpropagation, ²ANN-SNN Migration Learning Technique, ³Wilson, ⁴Gradient Descent, ⁵Intrinsic Plasticity, ⁶Hybrid Updating Algorithm, ⁷Grammatical Evolution, ⁸Astrocyte-Neuron Network, ⁹Multiplicative Gradient Descent, ¹⁰Tempotron Supervised Learning, ¹¹Hebbian Learning, ¹²FORCE Learning, ¹³Back Propagation Spike-Driven Learning, ¹⁴Error Gradient Back-Propagation, ¹⁵Spiking Neurons (RNeuron and WTANeuron and DelayNeuron), ¹⁶Converted SNNs, ¹⁷ANN-to-SNN Weight Learning (ANN-to-SNN Conversion), ¹⁸Hybrid SNN-ANN, ¹⁹Spiking Astrocyte Neural Network, ²⁰Spiking Competitive Winner-Take-All Network, ²¹Conversion, ²²Spiking Winner-Take-All (SWTA) Network;

***DSNN refers to Deep Spiking Neural Network; HSNN refers Hierarchical Spiking Neural Network; SELM refers to Spiking Extreme Learning Machine; CANN refers to Continuous Attractor Neural Network; SCNN refers to Spiking Convolutional Neural Network; RSNN refers to Recurrent Spiking Neural Network; CSNN refers to Convolutional Spiking Neural Network; PSNN refers to Polychronous Spiking Neural Network; HSN refers to Hippocampal Spiking Network; SDN refers to Spiking Deep Network; HNN refers to Hybrid Spiking and Convolutional Neural Network;

TABLE XVIII
SUMMARY OF ADVANTAGES AND DISADVANTAGES OF THE SELECTED
REVIEWED PAPERS

Ref.	Advantages	Disadvantages/Limitations
[41]	•Multiplierless Technique	•No Robustness to Noise
[63]	•CORDIC STDP with High Precision	•No Various CORDIC Algorithms
[64]	•Adaptive Clock/Event-Driven Computing •Robust to Noise	•No Applying Approximate Computing
[65]	•Parallel Computation with Low Bit-Width	•No Weights Binarization
[66]	•Real-Time Processing	•Constraint in Scaling up
[67]	•Time-Multiplexing Technique	•Limited to a Single-Chip System
[68]	•Autonomy of Learning in Hardware	•Not Scalable: High Bits Demand
[69]	•End-to-End Framework Based on RTL •Multiple Levels of Parallelism in Hardware	•High Latency and High Memory Usage by Long Spike Trains
[70]	•Fast and Low Bit-Width Precision CORDIC •Time-Division Multiplexing	•No Various CORDIC Types
[71]	•Parallel Processor Architectures	•High Parallelism: High Energy/Power
[72]	•Resource Sharing	•Network Size and Speed
[73]	•Fault-Tolerant Bio-Inspired System	•No Noisy Sensor Data
[74]	•Low Energy and Fast Convergence	•High Bit-Width
[75]	•Parallel and Pipeline Techniques	•Not Quite Complex for Visual Tasks
[76]	•Robust to Noise	•Dearth of Effective Training Methods
[77]	•Robustness and Anti-Noise Ability •Dynamic and Fully parallelized Architecture	•Lack of Employing On-FPGA Learning
[78]	•Genetic Algorithm to Find Optimal Parameters •Parallel Digital Architecture	•Genetic Algorithm Adds Complexity •No Various Optimization Algorithms
[83]	•Pipelined Dual-Core Architecture	•Limited Performance in Tough Tasks
[84]	•Improved Butterfly Fat Tree NoC Topology	•Need for Efficient Logic Partitioning
[86]	•Pipelined Multi-Kernel Convolution	•Computationally Costly Kernel Size
[87]	•Approximate Computing in Arithmetic Blocks	•Complex Learning Mechanism
[88]	•Reusable Arithmetic Hardware Units	•Lack of Adaptive Training Algorithm
[89]	•Deal with Multiple Data Flows	•Lack of Real-Time User Interface
[91]	•RRAM Crossbars with Pipelined Operations •RRAM Noise Models	•RRAM Device Variations: Static and Dynamic
[93]	•SMD Array Processor Architecture	•Relatively Low Clock Frequency
[94]	•Spatio-Temporal Sparsity	•Restricted Versatility to Spatio-Temporal Tasks
[95]	•Fault-Tolerant Capability in Hardware •Pipelining Strategy at the System Level	•Need for Efficient Hardware Interconnection and Optimization
[97]	•Robust to Noise	•Memory and Logic Overhead
[99]	•Combined Circuit for Complex Computing	•Efficient Hardware for a Simple Task
[102]	•Automate Neural Model Hardware Mapping •Customizability	•Limited Functionality in Complex Neural Dynamics Simulation
[103]	•Parallel SIMD Architecture	•No Area Consumption Optimization
[105]	•Fault-Tolerant NoC Router	•No Multicasting Routing Support
[107]	•Systolic Array to Control Data Flows	•Lack of a Big Array Organization
[108]	•Highly Parallel Hardware Accelerator	•Power Consumption, Lacking Power-Gating Technique
[112]	•Pipeline and CORDIC Computations	•No Multiple CORDIC Algorithms
[113]	•Real-Time Synchronous Pipelined Processing •Mixed Precision Quantization	•No Evaluation of the Accelerator for Neuromorphic Datasets
[114]	•Variable Precision Method for Low Bit-Width	•Not Applying Approximate Computing for Large-Scale SNNs
[117]	•Algorithm-Level Optimization	•Complicated Learning Mechanisms
[118]	•Generated Rhythmic Signals	•Less Compact Data Format
[119]	•Approximate Pipeline Hardware Design	•Compute- and Memory-Intensive
[120]	•Hierarchical Expandable Tree-Based Topology	•No STDP-Based On-Line Learning
[121]	•Parallel-Pipeline and Multiplexing Techniques	•No On-FPGA Unsupervised Learning
[122]	•PEs with Dynamic Partial Reconfiguration •Low Weight Bit-Width	•Complex Design Process •Complex Resource Management
[123]	•Principal Component Analysis IP Core	•Missing Logical Data Representation
[124]	•Fault-Tolerant Routing Scheme	•Simple Neural Model on Hardware
[127]	•Robust SNN Processor with Low Bit-Width	•No Tests on Complex Datasets
[129]	•Pruned SNN in a Parallel Digital Scheme	•No Evaluations on Complex Datasets
[132]	•Hardware-in-the-Loop Real-Time Computing	•Limited Respiratory Data
[134]	•Reliable and Fault-Tolerant Trojan Detection	•No Test on a Real-World Task
[136]	•Multiple Brain-Inspired Computing Paradigms	•Complex Learning with High Costs
[137]	•Quantization •Memory Interlacing Scheme	•Unproven Scalability for Processing Larger SNNs
[138]	•Low Bit-Width Hardware Acceleration	•No Partial Reconfiguration
[139]	•Multiple Neural Coding Schemes	•No Complex Vision Tasks
[141]	•Efficient Memory Access	•No Dataset Expansion
[142]	•Data Sparsity •Data Reuse and Parallel Data Access	•Restricted Versatility to Spatio-Temporal Tasks
[145]	•3-D Butterfly Fat Tree Topology	•Complexity in Generalization

TABLE XIX
LINKS TO OPEN-SOURCE CODES

Ref.	Links to Open-Source Codes
[15] ¹	https://github.com/SpiNNakerManchester
[17]	https://github.com/ChFrenkel/ODIN https://github.com/ChFrenkel/tinyODIN
[19]	https://github.com/electronicvisions
[69]	https://github.com/DanielGerlinghoff/radix-encoding
[75]	https://github.com/T-head-Semi/wujian100_open
[79]	https://ua-rcl.github.io/projects/ranc https://github.com/UA-RCL/RANC
[93]	upon e-mailed request to the corresponding author
[96]	https://github.com/YaribNevarez/sbs-framework.git
[113]	https://github.com/SFU-HiAccel/SyncNN
[116]	http://seis.bris.ac.uk/~eejlny/downloads/s2nn_package.zip https://github.com/eejlny/S2NN-HLS
[135]	https://github.com/saunak1994/CyNAPSEv11
[136]	https://github.com/Cqu-whb/TripleBrain.git

¹ **Note:** The link related to the reference [15] provides access to the software stacks associated with the SpiNNaker architecture. However, the SpiNNaker hardware design itself is not open source.

bit-width precision, CORDIC, multiplierless, parallel processing, and pipelining in FPGA acceleration have shown promising outcomes in achieving optimal reconfigurable hardware architectures for advanced neuromorphic and neural network systems.

Open-source codes provide reproducibility and knowledge sharing in research development, but it is not appealing for all researchers to make their codes publicly available, often due to concerns over intellectual property or the desire to maintain control over their works. In Table XIX, we have provided links to open-source codes that are explicitly mentioned in the corresponding reviewed papers.

IV. DISCUSSION AND FUTURE RESEARCH DIRECTIONS

Developing efficient algorithms and hardware, which are capable of simulating the human brain functionalities, is the heart of research in neuromorphic computing. The main challenge in the neuromorphic field is implementations of the large-scale neural networks like SNNs and deep neural networks [146]. The large-scale networks are time-consuming in terms of execution and hardware prototyping and require more silicon area with high power/energy consumption. Finding techniques to reduce occupied memory area, cost, power/energy, and enhance performance, while maintaining an acceptable level of error in various complex applications, is highly essential. On the other hand, implementations of SNNs and their learning algorithms for more complex tasks like video processing (comprising both image and speech processing), robotics, space application, medical implants, autonomous vehicles, etc. make a promising future to use FPGA hardware platforms as flexible, reprogrammable, and reconfigurable devices. Also, FPGAs benefit from a very fast development time and time to market with affordable costs.

In FPGA implementations of large-scale SNNs in intricate tasks, the number of spiking neurons and updating the synaptic weights incur high volume of computations that require the big size in the memory of the FPGAs. These issues also direct to consume more power/energy and reduce the processing speed.

fault-tolerant systems), the trade-off between low precision and efficiency in hardware designs, and the challenges of scaling parallel processing systems due to power consumption and neural network limitations. These trade-offs emphasize the need for careful balancing between efficiency, performance, precision, and scalability when designing efficient FPGA-based neuromorphic computing systems. The techniques such as low

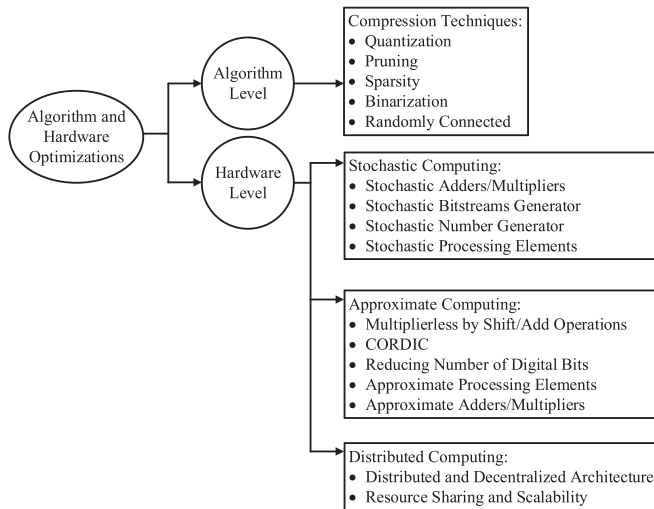


Fig. 9. The proposed categorization of optimization methods in algorithm and hardware designs.

As the memories in FPGAs do not have sufficient space to cover such computations, it is crucial to find ways to reduce the occupied memory area. Overall, considering FPGA design trade-offs in power, speed, and cost is essential for optimizing performance and efficiency. Balancing these factors ensures that the design meets application goals while maintaining high performance, low-power consumption, and cost-effectiveness. To this end, optimizations at the algorithm and hardware levels should be considered to speed up SNNs inference and diminish storage data. Fig. 9 provides an overview of the classification of various approaches for the algorithm- and hardware-level optimizations.

At the algorithm level, compression techniques like quantization and pruning are suggested [7], [11]. Pruning reduces the size of the SNN by removing the low-significance weights and neurons. The kernels of weights and overall SNN become sparse after pruning. Quantization reduces the precision of weights and activations by converting the values and operations from floating-point format into lower-bit floating-point, lower-bit integer, or lower-bit fixed-point data. The goal of these approaches is to approximately maintain the accuracy of the SNNs while improving energy efficiency and reducing memory footprint. This is particularly important for deploying neural networks on resource-constrained platforms, such as edge devices and mobile devices, where the computational and memory resources are limited.

At the hardware level, stochastic computing [7], [35], [40] and approximate computing [11], [71], [114] are the techniques that have been used in the neuromorphic research field, and more and more they need to be improved and considered in different aspects. Employing these computational techniques brings bright roads leading to low power/energy/area consumption and high performance in hardware implementations of large-scale SNNs.

Stochastic computing and approximate computing reduce the hardware complexity by simplifying the costly operations. Approximate computing imposes faults in flip-flops and digital

blocks. An analysis of the unsolicited faults in the digital hardware implementations of neuromorphic systems could be expressed as a challenging path for future. Although approximate computing makes inexact computations, a minor loss in precision and accuracy is acceptable. Stochastic computing processes the data in the format of random binary bit streams of 0s and 1s that interpret the data as probabilities. Accordingly, stochastic computing takes advantage of its powerful fault-tolerant feature. Moreover, extreme parallelism of stochastic computing makes it a competitive approach for resource-constrained hardware implementations. On the other hand, it is important to say that high precision in stochastic computing requires long bit streams. The long bit streams processing increases computation time and power consumption. So, this trade-off should be carefully handled in stochastic computing.

By employing the stochastic and approximate computing techniques, efficient hardware implementations of large-scale neuromorphic systems could be realized in parallel and scalable architectures for complicated tasks. Both techniques are still young and have the potential to be expanded and discussed as critical points in further research for industry and academia.

Distributed computing techniques include dividing a computational task across multiple processing units to enhance scalability, performance, and efficiency. Interconnecting multiple FPGAs to construct large-scale SNNs is prominent progress in the realm of neuromorphic computing. The computational processing capability of the sizeable SNNs can be greatly scaled up by linking multiple FPGAs in a distributed and decentralized manner. The nature of the parallelism and the reconfigurability of the computational logical units and memory blocks in FPGAs are very well suited to implement SNNs. However, because of constraints on hardware utilization resources, scaling up the SNNs to implement larger and more sophisticated networks on a single FPGA is challenging. To overcome these limitations, a distributed and decentralized architecture can be created through interconnecting of multiple FPGAs. This method leads to considerably scaling up the computational processing capabilities by sharing and combining the resources of several FPGAs.

To distribute the computational load during the entire system, each FPGA is dedicated to a group of neurons and synapses. High-speed interconnects are considered for data communication and transfer between FPGAs. Real-time processing and complex spiking neural modeling are effectively possible through such a distributed configuration. Also, the decentralized nature of the distributed setup is completely matched with the principles of neuromorphic computing, in which the computations are not centralized, and they are distributed across many connected processing units. Consequently, this technique improves the computational capacity and provides efficient real-time processing, enabling the distributed hardware architecture system closer to brain-inspired neuromorphic computing.

Emerging trends in FPGA-based neuromorphic computing are closely linked to the rise of Edge AI accelerators, which employ FPGAs for low-latency and real-time neural processing algorithms in autonomous systems and IoT devices. Moreover, hybrid ASIC-FPGA solutions in neuromorphic computing could

be attractive to deal with complex tasks as they combine the energy efficiency and speed of ASICs with the parallel processing capability and reconfigurability of FPGAs.

Contemporary research in the field of programmable logic devices is predominantly focused on general-purpose FPGA development. Notwithstanding their popularity, most FPGA development boards possess inherent limitations, including inadequate interfacing capabilities and restricted memory capacity, thereby rendering them a challenging proposition for many advanced applications. As a potential solution, the manufacturing of integrated circuit-level FPGAs is anticipated to facilitate seamless interfacing and enable wider memory utilization, thus overcoming the current limitations of FPGA development boards.

V. CONCLUSION

In this survey, energy-efficient and high-performance FPGA implementations of spiking neuron models and SNNs in various tasks have been reviewed. In neuromorphic computing, SNNs have been deemed as a brain-inspired computing paradigm, providing real-time and low-power solutions to challenging issues in digital hardware implementations. The biologically plausible computational neural models, the various structures of the SNNs, and the learning algorithms play significant roles to achieve acceptable accuracy. The applications such as classification, recognition, approximation, robotics, etc. were explored under different learning rules and neural structures. Hardware optimization techniques, including multiplierless designs, CORDIC designs, low bit-width computing, parallel processing, pipelined architectures, systolic array designs, robust fault tolerance, approximate computing, stochastic computing, and distributed computing are the significant methods to efficiently implement the SNNs on FPGAs. The development of digital reconfigurable architectures for SNNs is still in its infancy; therefore, additional research and hardware optimization approaches should be expected in order to improve power, energy, cost, speed, latency, and throughput.

According to the various analyses we provided, distinct categories of advantages and limitations have been identified in current implementations. The primary merits encompass performance optimization techniques such as computational efficiency, architecture optimization, and resource management. Neuromorphic systems capabilities have been improved through robustness features, advanced learning capabilities, and implementation flexibility. The demerits are presented in technical and functional domains. The technical limitations include performance constraints, scaling challenges, and implementation complexities. The functional restrictions consist of learning/training limitations and application constraints.

Our analysis and discussion reveal the distinct approaches for FPGA-based SNN implementations. These strategies address the challenges associated with neuromorphic computing. Through a thorough investigation of the literature, we found several significant trends and patterns in the implementations.

Looking to the future, certain aspects stand out and come to mind as paramount for further research. These include:

Designing and developing highly efficient scaling techniques for large-scale neuromorphic systems.

Investigating cutting-edge learning algorithms with consideration to hardware efficiency in terms of power, area, and speed.

Need to bridge the gap between theoretical capability and practical verification via extensive testing using intricate and real-world datasets.

Finding a compromise between resource optimization and system complexity.

Improvement of versatility in implementations to handle diverse neural computing tasks.

Long story short, and based on what has been discussed, although there has been substantial progression in FPGA-based neuromorphic computing systems, there are still considerable opportunities for advancements to create more scalable, efficient, and versatile implementations. Prospective innovations should concentrate on addressing these issues while maintaining the advantages of existing methods in performance and resource utilization. We believe that this comprehensive review provides a fundamental resource to smooth the path for researchers to better understand concepts and bring up original ideas for upcoming breakthroughs in the field.

REFERENCES

- [1] E. P. De Benedictis, "It's time to redefine Moore's law again," *Computer*, vol. 50, no. 2, pp. 72–75, Feb. 2017.
- [2] T. N. Theis and H.-P. Wong, "The end of Moore's law: A new beginning for information technology," *Comput. Sci. Eng.*, vol. 19, no. 2, pp. 41–50, Mar./Apr. 2017.
- [3] A. R. Young, M. E. Dean, J. S. Plank, and G. S. Rose, "A review of spiking neuromorphic hardware communication systems," *IEEE Access*, vol. 7, pp. 135606–135620, 2019.
- [4] C. D. Schuman et al., "A survey of neuromorphic computing and neural networks in hardware," 2017, arxiv.org/abs/1705.06963.
- [5] M. A. Talib, S. Majzoub, Q. Nasir, and D. Jamal, "A systematic literature review on hardware implementation of artificial intelligence algorithms," *J. Supercomput.*, vol. 77, pp. 1897–1938, May 2020.
- [6] M. Capra, B. Bussolino, A. Marchisio, G. Masera, M. Martina, and M. Shafique, "Hardware and software optimizations for accelerating deep neural networks: Survey of current trends, challenges, and the road ahead," *IEEE Access*, vol. 8, pp. 225134–225180, 2020.
- [7] L. Chen, X. Xiong, and J. Liu, "A survey of intelligent chip design research based on spiking neural networks," *IEEE Access*, vol. 10, pp. 89663–89686, 2022.
- [8] C. D. Schuman et al., "Opportunities for neuromorphic computing algorithms and applications," *Nature Comput. Sci.*, vol. 2, pp. 10–19, 2022.
- [9] A. Javanshir, T. T. Nguyen, M. A. P. Mahmud, and A. Z. Kouzani, "Advancements in algorithms and neuromorphic hardware for spiking neural networks," *Neural Computation*, vol. 34, no. 6, pp. 1289–1328, May 2022.
- [10] A. Shrestha, H. Fang, Z. Mei, D. P. Rider, Q. Wu, and Q. Qiu, "A survey on neuromorphic computing: Models and hardware," *IEEE Circuits Syst. Mag.*, vol. 22, no. 2, pp. 6–35, Secondquarter 2022.
- [11] M. Bouvier et al., "Spiking neural networks hardware implementations and challenges: A survey," *ACM J. Emerg. Technol. Comput. Syst.*, vol. 15, no. 2, pp. 1–35, Apr. 2019.
- [12] B. Rajendran, A. Sebastian, M. Schmuker, N. Srinivasa, and E. Eleftheriou, "Low-power neuromorphic hardware for signal processing applications: A review of architectural and system-level design approaches," *IEEE Signal Process. Mag.*, vol. 36, no. 6, pp. 97–110, Nov. 2019.
- [13] M. Davies et al., "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan./Feb. 2018.

- [14] F. Akopyan et al., "TrueNorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 34, no. 10, pp. 1537–1557, Oct. 2015.
- [15] S. B. Furber, F. Galluppi, S. Temple, and L. A. Plana, "The SpiNNaker project," *Proc. IEEE*, vol. 102, no. 5, pp. 652–665, May 2014.
- [16] C. Mayr et al., "Spinnaker 2: A 10 million core processor system for brain simulation and machine learning," 2019, *arXiv:1911.02385*.
- [17] C. Frenkel, M. Lefebvre, J. Legat, and D. Bol, "A 0.086-mm² 12.7-pJ/SOP 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm CMOS," *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 1, pp. 145–158, Feb. 2019.
- [18] B. V. Benjamin et al., "Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations," *Proc. IEEE*, vol. 102, no. 5, pp. 699–716, May 2014.
- [19] C. Pehle et al., "The BrainScale-S2 accelerated neuromorphic system with hybrid plasticity," *Front. Neurosci.*, vol. 16, Feb. 2022, Art. no. 795876.
- [20] S. Moradi, N. Qiao, F. Stefanini, and G. Indiveri, "A scalable multicore architecture with heterogeneous memory structures for dynamic neuromorphic asynchronous processors (DYNAPs)," *IEEE Trans. Biomed. Circuits Syst.*, vol. 12, no. 1, pp. 106–122, Feb. 2018.
- [21] L. Deng et al., "Tianjic: A unified and scalable chip bridging spike-based and continuous neural computation," *IEEE J. Solid-State Circuits*, vol. 55, no. 8, pp. 2228–2246, Aug. 2020.
- [22] A. Agrawal, M. Ali, M. Koo, N. Rathi, A. Jaiswal, and K. Roy, "IMPULSE: A 65-nm digital compute-in-memory macro with fused weights and membrane potential for spike-based sequential learning tasks," *IEEE Solid-State Circuits Lett.*, vol. 4, pp. 137–140, 2021.
- [23] J. Stuijt et al., "μBrain: An event-driven and fully synthesizable architecture for spiking neural networks," *Front. Neurosci.*, vol. 15, 2021, Art. no. 664208.
- [24] A. Neckar et al., "Braindrop: A mixed-signal neuromorphic architecture with a dynamical systems-based programming model," *Proc. IEEE*, vol. 107, no. 1, pp. 144–164, Jan. 2019.
- [25] C. Frenkel, J.-D. Legat, and D. Bol, "MorphIC: A 65-nm 738k-synapse/mm² quad-core binary-weight digital neuromorphic processor with stochastic spike-driven online learning," *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 5, pp. 999–1010, Oct. 2019.
- [26] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, "[DL] A survey of FPGA-based Neural Network inference accelerators," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 12, no. 1, pp. 1–26, 2019.
- [27] Z. Liu et al., "Throughput-optimized FPGA accelerator for deep convolutional neural networks," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 10, no. 3, pp. 1–23, Jul. 2017.
- [28] E. Roorda et al., "FPGA Architecture exploration for DNN Acceleration," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 15, no. 3, pp. 1–37, 2022.
- [29] C. Wu, M. Wang, X. Chu, K. Wang, and L. He, "Low-precision floating-point arithmetic for high-performance FPGA-based CNN acceleration," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 15, no. 1, pp. 1–21, 2021.
- [30] H. Soleimani, A. Ahmadi, and M. Bavandpour, "Biologically inspired spiking neurons: Piecewise linear models and digital implementation," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 59, no. 12, pp. 2991–3004, Dec. 2012.
- [31] S. Haghir, A. Zahedi, A. Naderi, and A. Ahmadi, "Multiplierless implementation of noisy izhikevich neuron with low-cost digital design," *IEEE Trans. Biomed. Circuits Syst.*, vol. 12, no. 6, pp. 1422–1430, Dec. 2018.
- [32] M. F. Tolba, A. H. Elsafty, M. Armanys, L. A. Said, A. H. Madian, and A. G. Radwan, "Synchronization and FPGA realization of fractional-order Izhikevich neuron model," *Microelectron. J.*, vol. 89, pp. 56–69, Jul. 2019.
- [33] J. Pu, W. L. Goh, V. P. Nambiar, Y. S. Chong, and A. T. Do, "A low-cost high-throughput digital design of biorealistic spiking neuron," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 68, no. 4, pp. 1398–1402, Apr. 2021.
- [34] A. J. Leigh, M. Mirhassani, and R. Muscedere, "An efficient spiking neuron hardware system based on the hardware-oriented modified izhikevich neuron (HOMIN) model," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 67, no. 12, pp. 3377–3381, Dec. 2020.
- [35] M. A. Hedayatpour, M. A. Karami, and J. Shamsi, "Implementation of Izhikevich neuron based on stochastic computing using a novel inspired Omega-Flip stochastic number generator," *Int. J. Circuit Theory Appl.*, vol. 50, pp. 3104–3118, May 2022.
- [36] S. Gomar and A. Ahmadi, "Digital multiplierless implementation of biological adaptive-exponential neuron model," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 61, no. 4, pp. 1206–1219, Apr. 2014.
- [37] M. Heidarpour, A. Ahmadi, and R. Rashidzadeh, "A CORDIC based digital hardware for adaptive exponential integrate and fire neuron," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 63, no. 11, pp. 1986–1996, Nov. 2016.
- [38] E. Jokar, H. Abolfathi, A. Ahmadi, and M. Ahmadi, "An efficient uniform-segmented neuron model for large-scale neuromorphic circuit design: Simulation and FPGA Synthesis results," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 66, no. 6, pp. 2336–2349, Jun. 2019.
- [39] S. Haghir and A. Ahmadi, "A novel digital realization of AdEx neuron model," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 67, no. 8, pp. 1444–1448, Aug. 2020.
- [40] S. Xiao, W. Liu, Y. Guo, and Z. Yu, "Low-cost adaptive exponential integrate-and-fire neuron using stochastic computing," *IEEE Trans. Biomed. Circuits Syst.*, vol. 14, no. 5, pp. 942–950, Oct. 2020.
- [41] E. Z. Farsa, A. Ahmadi, M. A. Maleki, M. Gholami, and H. N. Rad, "A low-cost high-speed neuromorphic hardware based on spiking neural network," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 66, no. 9, pp. 1582–1586, Sep. 2019.
- [42] J. Kim, V. Kornijuk, C. Ye, and D. S. Jeong, "Hardware-efficient emulation of leaky integrate-and-fire model using template-scaling-based exponential function approximation," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 68, no. 1, pp. 350–362, Jan. 2021.
- [43] T. Hishiki and H. Torikai, "A novel rotate-and-fire digital spiking neuron and its neuron-like bifurcations and responses," *IEEE Trans. Neural Netw.*, vol. 22, no. 5, pp. 752–767, May 2011.
- [44] T. Matsubara, H. Torikai, and T. Hishiki, "A generalized rotate-and-fire digital spiking neuron model and its on-FPGA learning," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 58, no. 10, pp. 677–681, Oct. 2011.
- [45] M. Nouri, G. R. Karimi, A. Ahmadi, and D. Abbott, "Digital multiplierless implementation of the biological FitzHugh-Nagumo model," *Neurocomputing*, vol. 165, pp. 468–476, Oct. 2015.
- [46] A. Zahedi, S. Haghir, and M. Hayati, "Multiplierless digital implementation of time-varying FitzHugh-Nagumo model," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 66, no. 7, pp. 2662–2670, Jul. 2019.
- [47] M. Hayati, M. Nouri, D. Abbott, and S. Haghir, "Digital multiplierless realization of two-coupled biological Hindmarsh-Rose neuron model," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 63, no. 5, pp. 463–467, May 2016.
- [48] S. A. Malik and A. H. Mir, "Discrete multiplierless implementation of fractional order Hindmarsh-Rose model," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 5, no. 5, pp. 792–802, Oct. 2021.
- [49] M. Heidarpur, A. Ahmadi, and N. Kandalaft, "A digital implementation of 2D Hindmarsh-Rose neuron," *Nonlinear Dyn.*, vol. 89, no. 3, pp. 2259–2272, 2017.
- [50] J. Cai, H. Bao, M. Chen, Q. Xu, and B. Bao, "analog/digital multiplierless implementations for nullcline-characteristics-based piecewise linear hindmarsh-rose neuron model," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 69, no. 7, pp. 2916–2927, Jul. 2022.
- [51] M. Hayati, M. Nouri, S. Haghir, and D. Abbott, "Digital multiplierless realization of two coupled biological morris-lecar neuron model," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 62, no. 7, pp. 1805–1814, Jul. 2015.
- [52] A. Ghiasi and A. Zahedi, "Field-programmable gate arrays-based Morris-Lecar implementation using multiplierless digital approach and new divider-exponential modules," *Comput. Elect. Eng.*, vol. 99, 2022, Art. no. 107771.
- [53] A. J. Leigh, M. Heidarpur, and M. Mirhassani, "A high-accuracy digital implementation of the Morris-Lecar neuron with variable physiological parameters," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 69, no. 10, pp. 4138–4142, Oct. 2022.
- [54] F. Shama, S. Haghir, and M. A. Imani, "FPGA realization of hodgkin-huxley neuronal model," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 28, no. 5, pp. 1059–1068, May 2020.
- [55] S. Haghir, A. Naderi, B. Ghanbari, and A. Ahmadi, "high speed and low digital resources implementation of Hodgkin-Huxley neuronal model using base-2 functions," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 68, no. 1, pp. 275–287, Jan. 2021.
- [56] M. Amiri et al., "Digital realization of the proposed linear model of the Hodgkin-Huxley neuron," *Int. J. Circuit Theory Appl.*, vol. 47, no. 3, pp. 483–497, Mar. 2019.

- [57] M. A. Imani, A. Ahmadi, M. RadMalekshahi, and S. Haghighi, "Digital multiplierless realization of coupled wilson neuron model," *IEEE Trans. Biomed. Circuits Syst.*, vol. 12, no. 6, pp. 1431–1439, Dec. 2018.
- [58] M. Nouri, M. Hayati, T. Serrano-Gotarredona, and D. Abbott, "A Digital neuromorphic realization of the 2-D Wilson Neuron model," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 66, no. 1, pp. 136–140, Jan. 2019.
- [59] S. Gomar and M. Ahmadi, "Digital hardware implementation of Gaussian Wilson–Cowan neocortex model," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 3, no. 1, pp. 24–35, Feb. 2019.
- [60] H. Soleimani and E. M. Drakakis, "An efficient and reconfigurable synchronous neuron model," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 65, no. 1, pp. 91–95, Jan. 2018.
- [61] E. Jokar, H. Abolfathi, and A. Ahmadi, "A novel nonlinear function evaluation approach for efficient FPGA mapping of neuron and synaptic plasticity models," *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 2, pp. 454–469, Apr. 2019.
- [62] M. Balubaid et al., "Central nervous system: Overall considerations based on hardware realization of digital spiking silicon neurons (DSSNs) and synaptic coupling," *Mathematics*, vol. 10, no. 6, 2022, Art. no. 882.
- [63] M. Heidarpur, A. Ahmadi, M. Ahmadi, and M. R. Azghadi, "CORDIC-SNN: On-FPGA STDP learning with Izhikevich neurons," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 66, no. 7, pp. 2651–2661, Jul. 2019.
- [64] S. Li, Z. Zhang, R. Mao, J. Xiao, L. Chang, and J. Zhou, "A fast and energy-efficient SNN processor with adaptive clock/event-driven computation scheme and online learning," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 68, no. 4, pp. 1543–1552, Apr. 2021.
- [65] X. Ju, B. Fang, R. Yan, X. Xu, and H. Tang, "An FPGA implementation of deep spiking neural networks for low-power and fast classification," *Neural Computation*, vol. 32, no. 1, pp. 182–204, Jan. 2020.
- [66] D. Pani, P. Meloni, G. Tuveri, F. Palumbo, P. Massobrio, and L. Raffo, "An FPGA platform for real-time simulation of spiking neuronal networks," *Front. Neurosci.*, vol. 11, Feb. 2017, Art. no. 90.
- [67] D. Ma et al., "Darwin: A neuromorphic hardware co-processor based on spiking neural networks," *J. Syst. Archit.*, vol. 77, pp. 43–51, Jun. 2017.
- [68] H. Asgari, B. M.-N. Maybodi, R. Kreiser, and Y. Sandamirskaya, "Digital Multiplier-less spiking neural network architecture of reinforcement learning in a context-dependent task," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 10, no. 4, pp. 498–511, Dec. 2020.
- [69] D. Gerlinghoff, Z. Wang, X. Gu, R. S. M. Goh, and T. Luo, "E3NE: An end-to-end framework for accelerating spiking neural networks with emerging neural encoding on FPGAs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 3207–3219, Nov. 2022.
- [70] J. Wu et al., "Efficient design of spiking neural network with STDP learning based on fast CORDIC," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 68, no. 6, pp. 2522–2534, Jun. 2021.
- [71] Q. Wang, Y. Li, B. Shao, S. Dey, and P. Li, "Energy efficient parallel neuromorphic architectures with approximate arithmetic on FPGA," *Neurocomputing*, vol. 221, pp. 146–158, Jan. 2017.
- [72] S. Y. Bonabi, H. Asgharian, S. Safari, and M. N. Ahmadabadi, "FPGA implementation of a biological neural network based on the Hodgkin-Huxley neuron model," *Front. Neurosci.*, vol. 8, 2014, Art. no. 379.
- [73] A. P. Johnson et al., "Homeostatic fault tolerance in spiking neural networks: A dynamic hardware perspective," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 65, no. 2, pp. 687–699, Feb. 2018.
- [74] H. Asgari, B. M.-N. Maybodi, M. Payvand, and M. R. Azghadi, "Low-energy and fast spiking neural network for context-dependent learning on FPGA," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 67, no. 11, pp. 2697–2701, Nov. 2020.
- [75] Y. Liu, Y. Chen, W. Ye, and Y. Gui, "FPGA-NHAP: A General FPGA-based neuromorphic hardware acceleration platform with high speed and low power," *IEEE Trans. Circuits Syst. I: Reg. Papers*, vol. 69, no. 6, pp. 2553–2566, Jun. 2022.
- [76] D. Neil and S.-C. Liu, "Minitaur, an event-driven FPGA-based spiking neural network accelerator," *IEEE Trans. Very Large Scale Integration (VLSI) Syst.*, vol. 22, no. 12, pp. 2621–2628, Dec. 2014.
- [77] B. Deng, Y. Fan, J. Wang, and S. Yang, "Reconstruction of a fully parallelized auditory spiking neural network and FPGA implementation," *IEEE Trans. Biomed. Circuits Syst.*, vol. 15, no. 6, pp. 1320–1331, Dec. 2021.
- [78] W. Guo, H. E. Yantir, M. E. Fouda, A. M. Eltawil, and K. N. Salama, "Toward the optimal design and FPGA implementation of spiking neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 8, pp. 3988–4002, Aug. 2022.
- [79] J. Mack et al., "RANC: Reconfigurable architecture for neuromorphic computing," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 40, no. 11, pp. 2265–2278, Nov. 2021.
- [80] G. Karimi, M. Gholami, and E. Z. Farsa, "Digital implementation of biologically inspired Wilson model population behavior and learning," *Int. J. Circuit Theory Appl.*, vol. 46, no. 4, pp. 965–977, Apr. 2018.
- [81] M. Gholami, E. Z. Farsa, and G. Karimi, "Reconfigurable field-programmable gate array-based on-chip learning neuromorphic digital implementation for nonlinear function approximation," *Int. J. Circuit Theory Appl.*, vol. 49, no. 8, pp. 2425–2435, Jun. 2021.
- [82] E. Z. Farsa, S. Nazari, and M. Gholami, "Function approximation by hardware spiking neural network," *J. Comput. Electron.*, vol. 14, pp. 707–716, 2015.
- [83] Z. He et al., "A low-cost FPGA implementation of spiking extreme learning machine with on-chip reward-modulated STDP learning," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 69, no. 3, pp. 1657–1661, Mar. 2022.
- [84] S. Yang et al., "Scalable digital neuromorphic architecture for large-scale biophysically meaningful neural network with multi-compartment neurons," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 1, pp. 148–162, Jan. 2020.
- [85] H. You and K. Zhao, "Neuromorphic implementation of a continuous attractor neural network with various synaptic dynamics," *IEEE Access*, vol. 9, pp. 109224–109240, 2021.
- [86] J. Zhang, L. Feng, T. Wang, W. Shi, Y. Wang, and G. Zhang, "FPGA-based implementation of an event-driven spiking multi-kernel convolution architecture," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 69, no. 3, pp. 1682–1686, Mar. 2022.
- [87] Y. Liu, W. Zhang, and P. Li, "Enabling non-hebbian learning in recurrent spiking neural processors with hardware-friendly on-chip intrinsic plasticity," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 3, pp. 465–474, Sep. 2019.
- [88] J. Zhang, R. Wang, X. Pei, D. Luo, S. Hussain, and G. Zhang, "A fast spiking neural network accelerator based on BP-STDP algorithm and weighted neuron model," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 69, no. 4, pp. 2271–2275, Apr. 2022.
- [89] S. Yang et al., "Real-time neuromorphic system for large-scale conductance-based spiking neural networks," *IEEE Trans. Cybern.*, vol. 49, no. 7, pp. 2490–2503, Jul. 2019.
- [90] J. Han, Z. Li, W. Zheng, and Y. Zhang, "Hardware implementation of spiking neural networks on FPGA," *Tsinghua Sci. Technol.*, vol. 25, no. 4, pp. 479–486, Aug. 2020.
- [91] T. Luo et al., "An FPGA-based hardware emulator for neuromorphic chip with RRAM," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 39, no. 2, pp. 438–450, Feb. 2020.
- [92] E. I. Guerra-Hernandez, A. Espinal, P. Batres-Mendoza, C. H. Garcia-Capulin, R. De J. Romero-Troncoso, and H. Rostro-Gonzalez, "A FPGA-based neuromorphic locomotion system for Multi-legged robots," *IEEE Access*, vol. 5, pp. 8301–8312, 2017.
- [93] M. J. Pearson et al., "Implementing spiking neural networks for real-time signal-processing and control applications: A model-validated FPGA approach," *IEEE Trans. Neural Netw.*, vol. 18, no. 5, pp. 1472–1487, Sep. 2007.
- [94] Q. Chen, C. Gao, X. Fang, and H. Luan, "Skydiver: A spiking neural network accelerator exploiting spatio-temporal workload balance," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 41, no. 12, pp. 5732–5736, Dec. 2022.
- [95] J. Liu, J. Harkin, L. P. Maguire, L. J. McDaid, and J. J. Wade, "SPAN-NER: A self-repairing spiking neural network hardware architecture," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 4, pp. 1287–1300, Apr. 2018.
- [96] Y. Nevarez, D. Rotermund, K. R. Pawelzik, and A. Garcia-Ortiz, "Accelerating spike-by-spike neural networks on FPGA with hybrid custom floating-point and logarithmic dot-product approximation," *IEEE Access*, vol. 9, pp. 80603–80620, 2021.
- [97] R. Wang et al., "An FPGA implementation of a polychronous spiking neural network with delay adaptation," *Front. Neurosci.*, vol. 7, 2013, Art. no. 14.
- [98] Q. Y. Sun et al., "A spiking neural network for extraction of features in colour opponent visual pathways and FPGA implementation," *Neurocomputing*, vol. 228, pp. 119–132, 2017.
- [99] G. Zhang et al., "A low-cost and high-speed hardware implementation of spiking neural network," *Neurocomputing*, vol. 382, pp. 106–115, Mar. 2020.
- [100] A. Upegui, C. P. Reyes, and E. Sanchez, "An FPGA platform for on-line topology exploration of spiking neural networks," *Microprocess. Microsyst.*, vol. 29, pp. 211–223, 2005.

- [101] K. Akbarzadeh-Sherbaf et al., "A digital hardware implementation of spiking neural networks with binary FORCE training," *Neurocomputing*, vol. 412, pp. 129–142, 2020.
- [102] K. Cheung, S. R. Schultz, and W. Luk, "NeuroFlow: A general purpose spiking neural network simulation platform using customizable processors," *Front. Neurosci.*, vol. 9, Jan. 2016, Art. no. 516.
- [103] A. Sripad et al., "SNAVA-A real-time multi-FPGA multi-model spiking neural network simulation architecture," *Neural Netw.*, vol. 97, pp. 28–45, Jan. 2018.
- [104] S. Yang et al., "FPGA implementation of hippocampal spiking network and its real-time simulation on dynamical neuromodulation of oscillations," *Neurocomputing*, vol. 282, pp. 262–276, Mar. 2018.
- [105] S. Carrillo et al., "Advancing interconnect density for spiking neural network hardware implementations using traffic-aware adaptive network-on-chip routers," *Neural Netw.*, vol. 33, pp. 42–57, Sep. 2012.
- [106] J. Wang et al., "Spike-event X-ray image classification for 3D-NoC-based neuromorphic pneumonia detection," *Electronics*, vol. 11, no. 24, pp. 1–17, 2022.
- [107] S.-Q. Wang et al., "SIES: A novel implementation of spiking convolutional neural network inference engine on field-programmable gate array," *J. Comput. Sci. Technol.*, vol. 35, no. 2, pp. 475–489, Mar. 2020.
- [108] E. Lemaire et al., "Synaptic activity and hardware footprint of spiking neural networks in digital neuromorphic systems," *ACM Trans. Embedd. Comput. Syst.*, vol. 21, no. 6, pp. 1–26, 2022.
- [109] L. Wan, J. Liu, J. Harkin, L. McDaid, and Y. Luo, "Layered tile architecture for efficient hardware spiking neural networks," *Microprocess. Microsyst.*, vol. 53, no. 1, pp. 21–32, 2017.
- [110] L. Miró-Amarante, F. Gómez-Rodríguez, A. Jiménez-Fernandez, and G. Jiménez-Moreno, "A spiking neural network for real-time spanish vowel phonemes recognition," *Neurocomputing*, vol. 226, pp. 249–261, 2017.
- [111] S. Pande et al., "Fixed latency on-chip interconnect for hardware spiking neural network architectures," *Parallel Comput.*, vol. 39, no. 9, pp. 357–371, Sep. 2013.
- [112] F. Khoyratabe, F. Grassia, S. Saighi, and T. Levi, "Optimized real-time biomimetic neural network on FPGA for bio-hybridization," *Front. Neurosci.*, vol. 13, pp. 377, Apr. 2019.
- [113] S. Panchapakesan, Z. Fang, and J. Li, "SyncNN: Evaluating and accelerating spiking neural networks on FPGAs," *ACM Trans. Reconfig. Technol. Syst.*, vol. 15, no. 4, pp. 1–27, Dec. 2022.
- [114] Y. Wang et al., "Energy efficient spiking neural network processing using approximate arithmetic units and variable precision weights," *J. Parallel Distrib. Comput.*, vol. 158, pp. 164–175, 2021.
- [115] L.-C. Caron et al., "Event management for large scale event-driven digital hardware spiking neural networks," *Neural Netw.*, vol. 45, pp. 83–93, Sep. 2013.
- [116] F. G. Sanchez and J. Nunez-Yanez, "Energy proportional streaming spiking neural network in a reconfigurable system," *Microprocessors Microsystems*, vol. 53, pp. 57–67, 2017.
- [117] Y. Liu, S. S. Yenamachintala, and P. Li, "Energy-efficient FPGA spiking neural accelerators with supervised and unsupervised spike-timing-dependent-plasticity," *ACM J. Emerg. Technol. Comput. Syst. (JETC)*, vol. 15, pp. 1–19, 2019.
- [118] H. Rostro-Gonzalez et al., "A CPG system based on spiking neurons for hexapod robot locomotion," *Neurocomputing*, vol. 170, pp. 47–54, Dec. 2015.
- [119] K. Akbarzadeh-Sherbaf, B. Abdoli, S. Safari, and A.-H. Vahabie, "A scalable FPGA architecture for randomly connected networks of Hodgkin-Huxley neurons," *Front. Neurosci.*, vol. 12, Oct. 2018, Art. no. 698.
- [120] J. Park, T. Yu, S. Joshi, C. Maier, and G. Cauwenberghs, "Hierarchical address event routing for reconfigurable large-scale neuromorphic systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2408–2422, Oct. 2017.
- [121] W. Ye, Y. Chen, and Y. Liu, "The implementation and optimization of neuromorphic hardware for supporting spiking neural networks with MLP and CNN topologies," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 42, no. 2, pp. 448–461, Feb. 2023.
- [122] H. Irmak et al., "A dynamic reconfigurable architecture for hybrid spiking and convolutional FPGA-based neural network designs," *J. Low Power Electron. Appl.*, vol. 11, no. 3, 2021, Art. no. 32.
- [123] S. Nazari et al., "Neuromorphic circuit based on the un-supervised learning of biologically inspired spiking neural network for pattern recognition," *Eng. Appl. Artif. Intell.*, vol. 116, pp. 1–16, 2022.
- [124] S. Yang, J. Wang, B. Deng, M. R. Azghadi, and B. Linares-Barranco, "Neuromorphic context-dependent learning framework with fault-tolerant spike routing," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 12, pp. 7126–7140, Dec. 2022.
- [125] B. Schrauwen, M. D'Haene, D. Verstraeten, and J. Van Campenhout, "Compact hardware liquid state machines on FPGA for real-time speech recognition," *Neural Netw.*, vol. 21, no. 2, pp. 511–523, 2008.
- [126] W. Sun et al., "Scalable implementation of hippocampal network on digital neuromorphic system towards brain-inspired intelligence," *Appl. Sci.*, vol. 10, no. 8, Apr. 2020, Art. no. 2857.
- [127] H. Zheng, Y. Guo, X. Yang, S. Xiao, and Z. Yu, "Balancing the cost and performance trade-offs in SNN processors," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 68, no. 9, pp. 3172–3176, Sep. 2021.
- [128] X. Hao, S. Yang, B. Deng, J. Wang, X. Wei, and Y. Che, "A CORDIC based real-time implementation and analysis of a respiratory central pattern generator," *Neurocomputing*, vol. 423, pp. 373–388, Jan. 2021.
- [129] W. Guo et al., "Towards efficient neuromorphic hardware: Unsupervised adaptive neuron pruning," *Electronics*, vol. 9, no. 7, pp. 1–15, 2020.
- [130] M. Losh and D. Llamocca, "A low-power spike-like neural network design," *Electronics*, vol. 8, no. 12, pp. 1–28, 2019.
- [131] K. Liu et al., "A hardware implementation of SNN-based spatio-temporal memory model," *Front. Neurosci.*, vol. 13, pp. 1–10, Aug. 2019.
- [132] A. Zbrzeski et al., "Bio-inspired controller on an FPGA applied to closed-loop diaphragmatic stimulation," *Front. Neurosci.*, vol. 10, pp. 1–14, Jun. 2016.
- [133] M. Chen, L. Zu, H. Wang, and F. Su, "FPGA-based real-time simulation platform for Large-Scale STN-GPe network," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 28, no. 11, pp. 2537–2547, Nov. 2020.
- [134] A. P. Johnson et al., "Bio-inspired approaches to safety and security in IoT-enabled cyber-physical systems," *Sensors*, vol. 20, no. 3, pp. 1–19, 2020.
- [135] S. Saha et al., "CyNAPSE: A low-power reconfigurable neural inference accelerator for spiking neural networks," *J. Signal Process. Syst.*, vol. 92, no. 9, pp. 907–929, 2020.
- [136] H. Wang et al., "TripleBrain: A compact neuromorphic hardware core with fast on-chip self-organizing and reinforcement spike-timing dependent plasticity," *IEEE Trans. Biomed. Circuits Syst.*, vol. 16, no. 4, pp. 636–650, Aug. 2022.
- [137] J. Sommer, M. A. Özkan, O. Keszoce, and J. Teich, "Efficient hardware acceleration of sparsely active convolutional spiking neural networks," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 41, no. 11, pp. 3767–3778, Nov. 2022.
- [138] Z. Li et al., "Efficiency analysis of artificial vs. Spiking neural networks on FPGAs," *J. Syst. Archit.*, vol. 133, pp. 1–8, 2022.
- [139] J. Zhang, R. Wang, T. Wang, J. Liu, S. Dang, and G. Zhang, "A configurable spiking convolution architecture supporting multiple coding schemes on FPGA," *IEEE Trans. Circuits Syst. II: Exp. Briefs*, vol. 69, no. 12, pp. 5089–5093, Dec. 2022.
- [140] G. Sanchez, J. Madrenas, and J. Cosp-Vilella, "Legion-based image segmentation by means of spiking neural networks using normalized synaptic weights implemented on a compact scalable neuromorphic architecture," *Neurocomputing*, vol. 352, pp. 106–120, 2019.
- [141] R. Tapiador-Morales, A. Linares-Barranco, A. Jimenez-Fernandez, and G. Jimenez-Moreno, "Neuromorphic LIF row-by-row multiconvolution processor for FPGA," *IEEE Trans. Biomed. Circuits Syst.*, vol. 13, no. 1, pp. 159–169, Feb. 2019.
- [142] Q. Chen, C. Gao, and Y. Fu, "Cerebron: A reconfigurable architecture for spatiotemporal sparse spiking neural networks," *IEEE Trans. Very Large Scale Integration (VLSI) Syst.*, vol. 30, no. 10, pp. 1425–1437, Oct. 2022.
- [143] B. Abdoli and S. Safari, "A hardware-friendly real-time implementation of the auditory attention based on a novel spiking winner-take-all network," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 42, no. 8, pp. 2566–2579, Aug. 2023.
- [144] G. Leone, L. Raffo, and P. Meloni, "A bandwidth-efficient emulator of biologically-relevant spiking neural networks on FPGA," *IEEE Access*, vol. 10, pp. 76780–76793, 2022.
- [145] S. Yang, J. Wang, N. Zhang, B. Deng, Y. Pang, and M. R. Azghadi, "CerebelluMorphic: Large-scale neuromorphic model and architecture for supervised motor learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 9, pp. 4398–4412, Sep. 2022.
- [146] P. Plagwitz, F. Hannig, J. Teich, and O. Keszoce, "To spike or not to spike? A quantitative comparison of SNN and CNN FPGA implementations," Jun. 2023, *arXiv:2306.12742*.



Edris Zaman Farsa received the B.Sc. and M.Sc. degrees in electronics engineering from Azad University, Arak Branch and Kermanshah Science and Research Branch, Iran, in 2009 and 2014, respectively. From 2014 to 2020, he was a lecturer and Researcher with the Department of Computer Engineering, Azad University (Sanandaj Branch), Sanandaj, Kurdistan, Iran. From April 2023 to June 2023, he was a Researcher with the Department of Microsystems Engineering (IMTEK), University of Freiburg, Freiburg, Germany. He is currently a Research Assistant with the Institute of Applied Microelectronics and Data Technology, Faculty of Computer Science and Electrical Engineering, University of Rostock, Rostock, Germany. His main research interests include hardware/software co-design of signal processing systems, FPGAs, approximate computing, neuromorphic computing, embedded AI, and edge AI. He was also a Reviewer for some IEEE journals like IEEE TRANSACTION ON CIRCUITS AND SYSTEMS II, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY, IEEE TRANSACTIONS ON BIOMEDICAL CIRCUITS AND SYSTEMS and IEEE conferences such as ISCAS and MWSCAS.



Arash Ahmadi (Senior Member, IEEE) received the B.Sc. degree in electronics from the Sharif University of Technology, Tehran, Iran, in 1993, the M.Sc. degree from Tarbiat Modares University, Tehran, in 1997, and the Ph.D. degree in electronics from the University of Southampton, Southampton, U.K., in 2008. From 2008 to 2010, he was a Fellow Researcher with the SpiNNaker project at the University of Southampton. From 2010 to 2014, he was an Associate Professor with the Electrical Engineering Department, Razi University, from 2014–2018, he was a Visiting Scholar and from 2018 to 2021, he was an Assistant Professor with the University of Windsor, Windsor, Canada. He is currently a faculty member with the Department of Electronics at Carleton University, Ottawa, Canada. His current research interests include neuromorphic, circuit design and optimization, AI-on-a-Chip, bio-inspired computing, and memristors.



Oliver Keszocze received the Diploma in applied mathematics and the B.Sc. degree in computer science in 2011 and the Doctoral degree in 2017 from the University of Bremen, Bremen, Germany. Since 2014, he has also been a Researcher with the German Research Center for Artificial Intelligence. From 2018 to 2024 he was with the Friedrich-Alexander-University, Erlangen, Germany as the Chair for Hardware/Software Co-Design. Since 2024 he has been an Associate Professor with the Embedded Systems Engineering section, Denmark Technical University, Lyngby, Denmark. His research interests include several aspects of logic synthesis and computer-aided design in various application domains and for different technologies. His current research focuses on approximate computing for both ASICs and FPGAs. Mr. Keszocze is a TPC Member for several conferences, including DATE, ICCAD, and ASP-DAC. He is also a Reviewer for IEEE TCAD and several other journals. He is the Organizer of the Annual Special Session “Future Trends in Emerging Technologies” at the DSD Conference. He is glad to be able to support young researchers by being part of DATE conference’s Ph.D. Forum TPC.